

영상처리·검사 알고리즘

Threshold·Filter·Edge·Morphology·Blob·Inspection Pipeline

- | | |
|----|--|
| 01 | 9일차. 영상처리 기초 1: Histogram · Contrast · Threshold |
| 02 | 10일차. 영상처리 기초 2: Filtering · Gradient · Edge |
| 03 | 11일차. 영상처리 기초 3: Morphology · Connected Component · Blob |
| 04 | 16일차 A. 산업용 검사 알고리즘 선택과 측정 |
| 05 | 16일차 B. Inspection Pipeline: Acquisition에서 Result까지 |

원본 20개 Markdown의 내용·코드·표·수식을 보존하여 주제별로 편집

9일차. 영상처리 기초 1: Histogram · Contrast · Threshold

Phase 7-1 · 예상 학습 시간: 8~10시간 · 난이도: 입문→중급 · 중요도: ● 반드시 알아야 함

1. 이 Chapter에서 배우는 것

핵심 개념

- Gray Level과 Histogram
- Brightness와 Contrast의 차이
- Threshold와 Binary Image
- Foreground/Background Polarity
- Global, Otsu, Adaptive Threshold의 선택 기준
- ROI와 Mask를 이용한 제한적 처리
- Normalization과 Contrast Stretching
- Threshold Margin과 조명 변화 Robustness
- 8/12/16-bit Image에서 Threshold 의미

실무 목표

검사 Image의 Histogram과 ROI 통계를 보고 Manual/Otsu/Adaptive Threshold 중 적합한 방식을 선택한다. Binary 결과를 만든 뒤 Foreground 비율과 Blob/Edge 후속 처리를 고려하며, 한 장에서 잘 되는 Threshold가 아니라 Lot·위치·조명 변화에서도 분리가 유지되는 Parameter를 검증한다.

2. 선수 지식

- Chapter 2의 Gray Scale, Bit Depth, Dynamic Range
- Chapter 7의 조명 Contrast와 CNR
- ROI, 평균, 표준편차의 기초
- OpenCV `cv::Mat` , `CV_8UC1`

2.1 오늘의 Pipeline 위치

TEXT

Camera Image

↓

ROI 선택

↓

Gray / Brightness / Contrast 확인

↓

Histogram 분석

↓

Threshold

↓

Binary Image

↓

Morphology / Blob / Measurement / Judgement

Threshold는 최종 검사가 아니라 **Gray Image**를 **Object 후보**와 **Background 후보**로 나누는 중간 단계다.

3. 핵심 개념

3.1 Gray Level

8-bit Gray Image에서 각 Pixel은 일반적으로 **0~255** 값을 가진다.

FORMULA · CALCULATION

0 = Black
255 = White

하지만 Gray 값은 Object의 절대 속성이 아니다.

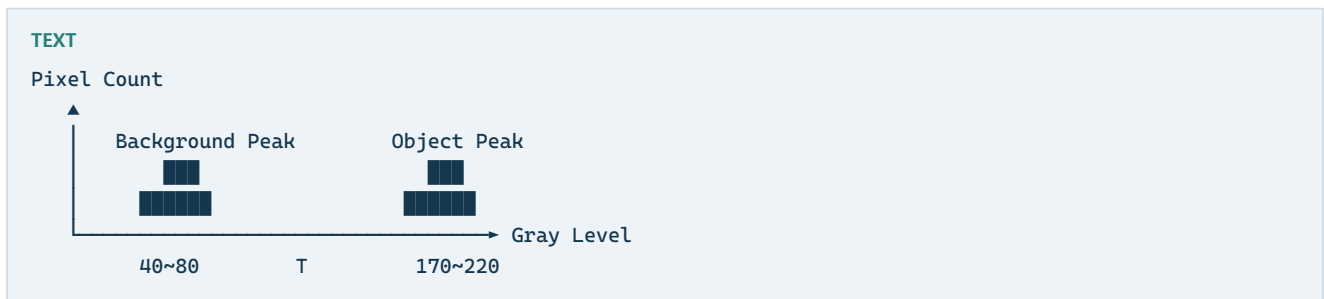
FORMULA · CALCULATION

Gray Level
= Object 반사/투과 특성
+ 조명 Geometry/세기
+ Lens/Aperture
+ Exposure/Gain
+ Sensor Response/Noise
+ Image Processing

같은 제품도 조명 온도, Exposure, 표면 오염과 위치 변화로 Gray Level이 달라질 수 있다.

3.2 Histogram

Histogram은 각 Gray Level 또는 구간에 Pixel이 몇 개 있는지 나타낸다.



두 Peak 사이 Valley에 Threshold **T** 를 놓으면 분리하기 쉽다. 그러나 다음 경우 Histogram만으로는 위치 정보를 알 수 없다.

- 다른 위치의 서로 다른 Object가 같은 Gray 값
- 넓은 조명 Gradient
- 작은 Defect가 전체 Histogram에서 거의 보이지 않음
- ROI 밖 Background가 대부분을 차지

따라서 전체 Image Histogram보다 검사 ROI와 정상/결함 Sample의 분포를 비교한다.

3.3 Brightness와 Contrast

- **Brightness:** Image 전체 또는 영역이 밝고 어두운 정도
- **Contrast:** 서로 다른 영역의 밝기 차이

FORMULA · CALCULATION

Image A: Background=50, Object=100 → Difference=50
Image B: Background=150, Object=200 → Difference=50

B가 전체적으로 밝지만 단순 평균 차이는 같다. 조명 Noise가 B에서 더 크다면 검사 성능은 다를 수 있으므로 CNR과 분포 겹침을 본다.

대표적인 단순 Contrast:

FORMULA · CALCULATION

Absolute Contrast = $|\mu_{\text{object}} - \mu_{\text{background}}|$
Michelson Contrast = $(I_{\text{max}} - I_{\text{min}}) / (I_{\text{max}} + I_{\text{min}})$

3.4 Threshold

Threshold는 Pixel 값을 기준으로 두 Class로 나눈다.

밝은 Object를 Foreground로 만들면:

TEXT

```
if pixel > T:
    output = 255
else:
    output = 0
```

어두운 Object를 Foreground로 만들면 반대로 Binary Inverse를 사용한다.

TEXT

```

if pixel <= T:
    output = 255
else:
    output = 0

```

Binary의 255는 “흰색으로 보인다”보다 **후속 처리에서 선택된 Foreground**라는 의미가 중요하다.

3.5 Global Manual Threshold

Image 또는 ROI 전체에 하나의 고정값 **T** 를 적용한다.

장점:

- 빠르고 해석하기 쉬움
- Recipe Parameter가 명확함
- 조명이 안정적이고 분포가 분리되면 매우 강건

단점:

- 밝기 Drift와 위치별 Shading에 민감
- Product Lot별 반사율 변화에 취약

잘 맞는 예:

- Back Light 외곽
- 안정된 Dark Field의 밝은 Particle
- 일정한 조명에서 Dark Mark/Print

3.6 Otsu Threshold

Otsu는 Histogram을 두 Class로 나눌 때 Class 내부 분산이 작아지는 Threshold를 자동 선택한다.

잘 맞는 조건:

- Object와 Background가 두 분포를 형성
- ROI 안에서 두 Class가 의미 있는 면적을 차지
- 조명 Gradient가 작음

주의할 조건:

- 작은 Defect가 Pixel의 극히 일부
- Background가 여러 재질/Pattern으로 구성
- Histogram이 단봉형
- 정상 Image와 불량 Image에서 선택 Threshold가 크게 달라짐

Otsu가 “최적 검사 Threshold”를 보장하지 않는다. 한 Image의 Histogram 분할 기준일 뿐 Spec과 검출률은 별도로 검증한다.

3.7 Adaptive Threshold

각 Pixel 주변의 Local Mean 또는 Gaussian Weighted Mean을 기준으로 Threshold를 계산한다.

FORMULA · CALCULATION

$$T(x,y) = \text{LocalMean}(x,y) - C$$

장점:

- 완만한 조명 Gradient와 Shading에 대응
- 문서/OCR/Mark처럼 Local Contrast가 중요한 경우

단점:

- Block Size 와 C 에 민감
- Texture/Noise를 Foreground로 만들 수 있음
- 큰 Defect와 작은 Window가 충돌할 수 있음
- Parameter 의미가 Global Threshold보다 직관적이지 않음

Block Size는 검출하려는 Feature보다 충분히 큰 주변 Background를 포함해야 하며 홀수여야 한다.

3.8 Threshold 방식 비교

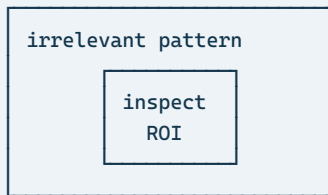
방식	기준	장점	약점	대표 적용
Manual Global	Recipe의 고정 T	빠름, 재현·해석 쉬움	Brightness Drift	Back Light, 안정 조명
Otsu	현재 Histogram	Parameter 감소	Class 비율/분포 의존	두 Peak가 명확한 분할
Adaptive Mean	Local 평균-C	Shading 대응	Texture/Noise	Print, 불균일 배경
Adaptive Gaussian	Local 가중 평균-C	중심 주변 반영	계산량/Parameter	Local Mark/문자
Range/InRange	$\text{Low} \leq \text{pixel} \leq \text{High}$	중간 밝기 선택	범위 관리 필요	특정 Gray/Color Band

3.9 ROI와 Mask

- **Rectangular ROI**: Image의 사각형 일부를 처리
- **Mask**: 임의 형태의 유효 Pixel만 선택

TEXT

Full Image



ROI를 사용하면:

- Histogram이 검사 대상 분포를 더 잘 반영
- False Positive 감소
- 처리 시간 감소
- Recipe 의도가 명확

Align 전 고정 ROI를 사용하면 제품 이동 시 실제 부위가 벗어날 수 있다. Align 후 Transform된 ROI에서 Threshold를 수행한다.

3.10 Normalization

Min-Max Normalization:

FORMULA · CALCULATION

$$\text{output} = (\text{input} - \text{min}) / (\text{max} - \text{min}) \times 255$$

장점:

- 좁은 Gray 범위를 Display/분석에 확대
- 일부 전처리에서 Contrast 확보

위험:

- Frame마다 Min/Max가 달라지면 동일 물체의 Gray 의미가 변함
- Noise/Outlier 하나가 범위를 결정
- 정상/불량 차이를 인위적으로 확대 또는 축소

검사에서는 고정 Calibration 범위, Percentile 기반 Stretch 또는 Reference 기반 보정을 고려한다. Review용 Normalize와 검사 원본을 분리한다.

3.11 Threshold Margin

정상 Background 최대값과 Object 최소값 사이 간격을 본다.

TEXT

Background distribution: 50~90
Object distribution: 150~210

Valid Threshold interval: $90 < T < 150$
Margin width: 60 Gray Level

한 Sample이 아니라 여러 Lot, 위치, 시간의 분포를 겹쳐야 한다.

FORMULA · CALCULATION

Worst Background Max = 130
Worst Object Min = 140
Available Margin = 10

Margin이 작으면 Threshold 미세 조정보다 조명과 촬영 조건 개선이 우선일 수 있다.

3.12 Bit Depth와 Threshold

8-bit의 Threshold 128을 12-bit Image에 그대로 사용하면 의미가 다르다.

전체 범위의 50% 기준:

FORMULA · CALCULATION

8-bit: $255 \times 0.5 \approx 128$
12-bit: $4095 \times 0.5 \approx 2048$
16-bit: $65535 \times 0.5 \approx 32768$

10/12-bit가 16-bit Container에 저장되는 경우 유효 Bit 정렬과 Black Level도 확인한다.

3.13 Binary 결과의 품질

Threshold 이후 다음을 확인한다.

- Foreground가 검사 Object와 일치하는가?
- Object 내부 Hole/Noise가 생겼는가?
- 서로 붙어야 할 영역이 끊겼는가?
- 서로 다른 Object가 붙었는가?
- FOV 가장자리/ROI 경계가 False Blob이 되었는가?
- Foreground 비율이 정상 범위인가?

다음 단계의 Morphology는 작은 결함을 지울 수도 있으므로 Threshold 원본과 전처리 결과를 함께 Review한다.

4. 그림으로 이해하기

4.1 Gray에서 Binary로

FORMULA · CALCULATION

Gray: [20 45 80 125 160 210 250]

T=128

Binary: [0 0 0 0 255 255 255]

4.2 조명 Gradient에서 Global Threshold 실패

TEXT

Left side dark Right side bright

Background: 40 140

Object: 100 200

Global T=120:

왼쪽 Object 일부 누락 + 오른쪽 Background 일부 검출

4.3 Align과 ROI의 연결

TEXT

Reference ROI

↓ Align X/Y/Theta

Transformed ROI

↓ Histogram / Threshold

Binary Feature

↓ Blob / Measurement

5. 실제 검사 장비에서 어디에 사용하는가?

Back Light 외곽 검사

밝은 Background와 어두운 제품을 `THRESH_BINARY_INV` 로 분리해 Contour와 Width를 측정한다. Edge가 포화/Blur되지 않도록 Exposure와 광원 Geometry를 조정한다.

Dark Field Particle 검사

어두운 Background 위 밝은 Particle을 Threshold한 뒤 작은 Blob의 Area와 밝기를 측정한다. Background Texture와 Sensor Noise가 False Blob이 되지 않도록 Local Contrast와 크기 조건을 함께 사용한다.

Mark/Print Presence

Align된 ROI에서 Dark Mark를 Binary로 분리하고 Foreground Area 또는 Pattern 특징으로 누락을 판정한다. ROI Alignment 없이 Threshold하면 위치 편차가 면적 변화처럼 보일 수 있다.

Stain/불균일 검사

Global Threshold보다 Background Estimation 또는 Adaptive/Local Difference가 유리할 수 있다. 넓고 약한 얼룩은 단순 Pixel Threshold보다 Reference Difference와 통계 검사가 필요하다.

Recipe Monitoring

Foreground Ratio, ROI Mean, Standard Deviation, Otsu Threshold 값을 Result에 기록하면 조명 Drift와 제품 분포 변화를 조기에 찾을 수 있다.

6. 숫자로 이해하기

예제 1: Threshold Margin

여러 정상 Sample에서:

FORMULA · CALCULATION

Background: mean=70, stddev=8
Object: mean=170, stddev=10

각 분포의 $\text{mean} \pm 3\sigma$ 를 보수적 범위로 보면:

FORMULA · CALCULATION

Background upper $\approx 70 + 3 \times 8 = 94$
Object lower $\approx 170 - 3 \times 10 = 140$

Threshold interval: $94 < T < 140$
Margin width: 46

정규분포 가정과 독립 Sample 조건을 확인해야 하며 실제 Worst Case 분포도 함께 본다.

예제 2: Min-Max Normalization

ROI Gray 범위가 50~200이고 Pixel 값이 100이라면:

FORMULA · CALCULATION

$(100-50)/(200-50) \times 255$
 $= 50/150 \times 255$
 $= 85$

다음 Frame의 범위가 80~180이면 같은 입력 100은:

FORMULA · CALCULATION

$$(100-80)/(180-80) \times 255 = 51$$

Frame별 Min-Max Normalize가 절대 Threshold 의미를 바꾸는 예다.

예제 3: Foreground Ratio

ROI가 $400 \times 300 = 120,000$ Pixel 이고 Binary Foreground가 18,600 Pixel이면:

FORMULA · CALCULATION

$$\begin{aligned} \text{Foreground Ratio} &= 18,600 / 120,000 \times 100 \\ &= 15.5\% \end{aligned}$$

정상 범위가 14~17%라면 15.5%는 범위 안이지만 Blob 분리 상태와 위치도 확인해야 한다.

예제 4: Bit Depth Threshold 변환

8-bit Threshold 180과 같은 전체 범위 비율을 12-bit로 변환하면:

FORMULA · CALCULATION

$$\begin{aligned} \text{Normalized ratio} &= 180 / 255 \approx 0.705882 \\ \text{12-bit threshold} &= 0.705882 \times 4095 \approx 2891 \end{aligned}$$

Gamma, Black Level과 Sensor Response가 다르면 단순 비례값이 동일한 광학 조건을 보장하지 않는다.

7. C++ 구현

C++17/OpenCV Threshold Pipeline

CPP · 1/2

```
#include <opencv2/opencv.hpp>

#include <cmath>
#include <limits>
#include <stdexcept>

enum class ThresholdMode {
    Manual,
    Otsu,
    AdaptiveMean,
    AdaptiveGaussian
};

struct ThresholdRecipe final {
    ThresholdMode mode{ThresholdMode::Manual};
    double manualThreshold{128.0};
    bool inverse{false};
    int gaussianKernel{1};
    int adaptiveBlockSize{31};
    double adaptiveC{5.0};
};

struct ThresholdResult final {
    cv::Mat binary;
    double usedGlobalThreshold{};
    double foregroundRatio{};
};

[[nodiscard]] ThresholdResult ApplyThreshold(
    const cv::Mat& gray,
    const ThresholdRecipe& recipe)
{
    if (gray.empty() || gray.type() != CV_8UC1) {
        throw std::invalid_argument{"CV_8UC1 image is required"};
    }
    if (recipe.gaussianKernel <= 0 || recipe.gaussianKernel % 2 == 0) {
        throw std::invalid_argument{"Gaussian kernel must be positive and odd"};
    }

    cv::Mat filtered;
    if (recipe.gaussianKernel == 1) {
        filtered = gray;
    } else {
        cv::GaussianBlur(gray, filtered,
                        cv::Size{recipe.gaussianKernel, recipe.gaussianKernel},
                        0.0);
    }
}
```

CPP · 2/2

```

cv::Mat binary;
double usedThreshold = std::numeric_limits<double>::quiet_NaN();
const int baseType = recipe.inverse ? cv::THRESH_BINARY_INV
                                   : cv::THRESH_BINARY;

switch (recipe.mode) {
case ThresholdMode::Manual:
    if (!std::isfinite(recipe.manualThreshold) ||
        recipe.manualThreshold < 0.0 || recipe.manualThreshold > 255.0) {
        throw std::invalid_argument{"Manual threshold must be in [0,255]"};
    }
    usedThreshold = cv::threshold(filtered, binary,
                                recipe.manualThreshold, 255.0, baseType);
    break;

case ThresholdMode::Otsu:
    usedThreshold = cv::threshold(filtered, binary, 0.0, 255.0,
                                baseType | cv::THRESH_OTSU);
    break;

case ThresholdMode::AdaptiveMean:
case ThresholdMode::AdaptiveGaussian:
    if (recipe.adaptiveBlockSize <= 1 ||
        recipe.adaptiveBlockSize % 2 == 0 ||
        recipe.adaptiveBlockSize > std::min(gray.rows, gray.cols)) {
        throw std::invalid_argument{
            "Adaptive block size must be odd and fit the image"};
    }
    cv::adaptiveThreshold(
        filtered, binary, 255.0,
        recipe.mode == ThresholdMode::AdaptiveMean
            ? cv::ADAPTIVE_THRESH_MEAN_C
            : cv::ADAPTIVE_THRESH_GAUSSIAN_C,
        baseType, recipe.adaptiveBlockSize, recipe.adaptiveC);
    break;
}

const double foregroundRatio =
    static_cast<double>(cv::countNonZero(binary)) /
    static_cast<double>(binary.total());

return {binary, usedThreshold, foregroundRatio};
}

```

Histogram 계산

CPP

```
[[nodiscard]] cv::Mat CalculateHistogram256(const cv::Mat& gray)
{
    if (gray.empty() || gray.type() != CV_8UC1) {
        throw std::invalid_argument{"CV_8UC1 image is required"};
    }

    constexpr int channel = 0;
    constexpr int bins = 256;
    const float range[] = {0.0F, 256.0F};
    const float* ranges[] = {range};
    cv::Mat histogram;

    cv::calcHist(&gray, 1, &channel, cv::Mat{}, histogram,
                1, &bins, ranges, true, false);
    return histogram;
}
```

Unit Test

C++

```
#include <cassert>
#include <cmath>

void TestManualThreshold()
{
    cv::Mat image(100, 100, CV_8UC1, cv::Scalar(50));
    image(cv::Rect{50, 0, 50, 100}).setTo(200);

    ThresholdRecipe recipe;
    recipe.mode = ThresholdMode::Manual;
    recipe.manualThreshold = 128.0;

    const auto result = ApplyThreshold(image, recipe);
    assert(std::abs(result.foregroundRatio - 0.5) < 1e-12);
    assert(result.usedGlobalThreshold == 128.0);
}

void TestOtsuSeparatesTwoLevels()
{
    cv::Mat image(100, 100, CV_8UC1, cv::Scalar(50));
    image(cv::Rect{50, 0, 50, 100}).setTo(200);

    ThresholdRecipe recipe;
    recipe.mode = ThresholdMode::Otsu;

    const auto result = ApplyThreshold(image, recipe);
    assert(std::abs(result.foregroundRatio - 0.5) < 1e-12);
    assert(std::isfinite(result.usedGlobalThreshold));
}

void TestInversePolarity()
{
    const cv::Mat image =
        (cv::Mat_<unsigned char>(1, 3) << 10, 100, 200);
    ThresholdRecipe recipe;
    recipe.manualThreshold = 128.0;
    recipe.inverse = true;

    const auto result = ApplyThreshold(image, recipe);
    assert(result.binary.at<unsigned char>(0, 0) == 255);
    assert(result.binary.at<unsigned char>(0, 2) == 0);
}
```

코드에서 봐야 할 점

1. 입력 Type을 `CV_8UC1` 로 제한해 Threshold 의미를 명확히 했다.
2. Manual/Otsu/Adaptive 방식과 Polarity를 Recipe로 분리했다.
3. Gaussian Kernel과 Adaptive Block Size를 사용 전 검증한다.
4. Adaptive 방식에는 단일 Global Threshold가 없으므로 `NaN` 으로 표현한다.
5. Binary와 함께 사용 Threshold 및 Foreground Ratio를 Result로 남긴다.
6. 실제 검사는 Align 후 ROI를 전달하고 원본/중간/결과 Image를 Review 가능하게 저장한다.

8. 실무에서 발생하는 문제

문제 1: 조명 열화로 고정 Threshold 실패

- 증상: 시간 경과에 따라 Foreground Area 감소
- 대응: ROI Mean/Histogram Monitoring, 조명 안정화, Threshold Margin 재검증

문제 2: Auto Exposure와 Otsu를 동시에 사용

- 증상: Frame마다 Image와 Threshold가 함께 변해 결과 원인 추적이 어려움
- 대응: Camera Parameter 고정, Otsu 값과 분포 기록, 허용 범위 설정

문제 3: 전체 Image Histogram 사용

- 증상: 큰 Background가 Histogram을 지배해 작은 Defect 분포가 사라짐
- 대응: Align된 검사 ROI와 Feature/Background Mask 사용

문제 4: Adaptive Block Size가 Feature보다 작음

- 증상: Object 내부가 Background처럼 사라지거나 Edge만 남음
- 대응: Feature와 조명 Gradient Scale을 기준으로 Window 설정

문제 5: Frame별 Normalize 후 고정 Threshold

- 증상: Outlier와 정상 Variation에 따라 동일 Gray가 다른 값으로 변환
- 대응: 검사 Normalize 기준 고정 또는 Reference 기반, Raw 통계 저장

문제 6: Foreground Polarity 반대

- 증상: Object가 아니라 Background 전체가 Blob으로 검출
- 대응: Recipe에 밝은/어두운 Object 정의, Foreground Ratio Sanity Check

문제 7: 12-bit Container 해석 오류

- 증상: Threshold 범위가 비정상이고 Histogram이 한쪽에 몰림
- 대응: Pixel Format, 유효 Bit, Alignment, Black Level 확인

9. 흔한 오해

현재 Histogram의 두 Class 분할값이지 검사 Spec 최적값이 아니다.

1. “Otsu는 항상 최적 Threshold를 찾는다.”

알고리즘이 Foreground로 선택한 Pixel일 뿐 Polarity에 따라 의미가 달라진다.

2. “Binary 255는 항상 실제 Object다.”

Noise, 분포 Tail, 위치별 Shading과 Lot Variation을 함께 봐야 한다.

3. “Contrast가 크면 Threshold는 항상 안정적이다.”

Texture와 Noise에 민감하고 Parameter가 늘어난다. 안정 조명에서는 Global이 더 단순하고 강건할 수 있다.

4. “Adaptive Threshold가 Global보다 고급이므로 더 좋다.”

값의 표현 범위를 바꿀 뿐 손실된 Optical Contrast를 복구하지 않는다.

5. “Normalization하면 원본 정보가 좋아진다.”

정상/불량 분포가 겹치면 조명 Geometry와 촬영 조건을 먼저 개선해야 한다.

6. “Threshold만 잘 조정하면 조명 문제를 해결한다.”

같은 면적이어도 위치와 Shape가 틀릴 수 있으므로 Blob Geometry를 확인한다.

7. “Foreground Ratio가 정상이라면 Binary도 정상이다.”

10. 면접에서 나올 수 있는 질문

Q1. Threshold는 검사 Pipeline에서 어떤 역할을 하나요?

초보자가 이해할 수 있는 답변 Gray Image를 기준 밝기보다 밝거나 어두운 두 그룹으로 나눠 검사할 Object 후보를 만듭니다.

실무자 답변 Threshold는 Gray Feature를 Binary Foreground/Background로 분할하는 Segmentation 단계다. 이후 Morphology, Blob, Contour와 Measurement가 사용한다. ROI, Polarity, Bit Depth와 조명 Variation을 포함해 Parameter Margin을 검증한다.

면접용 30초 답변 “Threshold는 Gray Image에서 검사 대상 후보를 Binary Foreground로 분리하는 단계입니다. Align된 ROI에 적용하고 Polarity와 Bit Depth를 명확히 하며, 여러 Lot의 Background 최대와 Object 최소 사이 Margin을 검증한 뒤 Blob/Measurement로 연결합니다.”

Q2. Manual과 Otsu Threshold의 차이는 무엇인가요?

초보자가 이해할 수 있는 답변 Manual은 사람이 고정값을 정하고 Otsu는 현재 Histogram을 두 그룹으로 잘 나누는 값을 자동 계산합니다.

실무자 답변 Manual은 Parameter 의미와 재현성이 명확하지만 Drift에 민감하다. Otsu는 Bimodal Histogram에 유리하지만 Object 비율과 분포 변화에 따라 값이 바뀐다. 선택 Threshold와 품질 분포를 Result에 기록한다.

면접용 30초 답변 “Manual은 Recipe에 고정 Threshold를 두어 빠르고 해석하기 쉽습니다. Otsu는 현재 Histogram의 Class 내 분산을 최소화해 자동 선택하지만 작은 Defect나 단봉 Histogram에는 부적합할 수 있습니다. 조명 안정성과 분포 형태로 선택합니다.”

Q3. Adaptive Threshold는 언제 사용하나요?

초보자가 이해할 수 있는 답변 Image 위치마다 밝기가 달라 하나의 Threshold로 분리하기 어려울 때 주변 밝기를 기준으로 사용합니다.

실무자 답변 완만한 Shading 아래 Local Mark/Print처럼 Feature Scale과 Background Scale이 분리될 때 사용한다. Block Size는 Feature보다 큰 Background 문맥을 포함해야 하며 Texture/Noise와 경계 Artifact를 검증한다.

면접용 30초 답변 "Global Threshold가 조명 Gradient 때문에 실패할 때 Local Mean 또는 Gaussian 기준의 Adaptive Threshold를 검토합니다. Block Size와 C가 Feature 크기보다 적절한지 확인하고 Texture를 False Foreground로 만들지 않는지 검증합니다."

Q4. Threshold가 조명 변화에 민감한 이유는 무엇인가요?

초보자가 이해할 수 있는 답변 조명이 바뀌면 같은 물체의 Pixel 값이 바뀌어 원래 Threshold의 반대쪽으로 이동할 수 있기 때문입니다.

실무자 답변 Threshold는 Absolute 또는 Local Gray 분포를 기준으로 한다. LED 온도/열화, Exposure, 표면 반사와 위치별 Shading이 Object/Background 분포를 이동·확장해 Tail이 겹친다. 조명 안정화와 CNR/Threshold Margin 관리가 우선이다.

면접용 30초 답변 "Pixel 값은 물체 고유값이 아니라 조명·Exposure·반사의 결과입니다. 변화가 Object와 Background 분포를 이동시켜 Threshold Margin을 줄입니다. 그래서 ROI Histogram과 분포를 Monitoring하고 광학 조건을 안정화한 뒤 Parameter를 정합니다."

Q5. ROI에서 Histogram을 봐야 하는 이유는 무엇인가요?

초보자가 이해할 수 있는 답변 전체 Image에는 검사와 관계없는 Background가 많아 실제 검사 부위의 밝기 분포를 가릴 수 있기 때문입니다.

실무자 답변 Global Histogram은 면적이 큰 무관 영역에 지배된다. Align된 ROI 또는 Mask에서 Feature/Background 분포를 측정해야 Threshold Margin과 CNR이 실제 검사 조건을 반영한다.

면접용 30초 답변 "작은 Defect는 전체 Image Histogram에서 거의 보이지 않고 큰 Background가 분포를 지배합니다. Align 후 검사 ROI와 Feature/Background Mask의 Histogram을 분리해 Threshold Margin과 Variation을 평가해야 합니다."

Q6. Normalization 사용 시 주의할 점은 무엇인가요?

초보자가 이해할 수 있는 답변 매 Image의 최소·최대값으로 바꾸면 같은 밝기의 Pixel도 Frame마다 다른 값이 될 수 있습니다.

실무자 답변 Frame별 Min-Max는 Outlier와 Sample 구성에 따라 Transfer Function이 변해 Fixed Threshold와 Image 간 비교를 깨뜨린다. Display와 Inspection 경로를 분리하고 고정/Reference/Percentile 기준의 Robustness를 검증한다.

면접용 30초 답변 "Normalization은 Contrast 표현을 넓히지만 Frame별 Min-Max를 쓰면 Gray의 절대 의미가 사라집니다. 검사에는 고정 또는 Reference 기반 Mapping을 사용하고 Raw 통계와 변환 Parameter를 함께 남기며 Display용 변환과 분리합니다."

11. 실습 문제

실습 1: Histogram과 Threshold Sweep

동일 ROI에서 Threshold를 50~220까지 5씩 변경하며 다음을 저장한다.

- Foreground Ratio
- Blob Count
- Largest Blob Area
- 정상/불량 판정

안정적으로 같은 결과를 내는 Threshold 구간을 찾는다.

실습 2: 조명 Drift 시뮬레이션

Gray Image에 `-30, -15, 0, +15, +30` Offset을 적용하고 Manual/Otsu/Adaptive 결과를 비교한다. 각 방식의 Foreground Area 변화와 실패 유형을 기록한다.

실습 3: ROI와 전체 Histogram 비교

작은 Defect가 있는 Image에서 전체 Histogram과 Defect 주변 ROI Histogram을 비교한다. 작은 결함 분포가 전체 Histogram에서 보이지 않는 이유를 Pixel 면적 비율로 설명한다.

실습 4: 12-bit Threshold

12-bit 유효 Image에서 Histogram, Threshold, 8-bit Display 변환을 분리한다. 유효 범위가 0~4095인지 MSB 정렬인지 확인하고 동일 비율 Threshold를 계산한다.

Phase 7-1 미니 프로젝트: Threshold Inspection Workbench

TEXT

```

Image Load
↓
ROI/Mask 선택
↓
Histogram + Mean/StdDev
↓
Manual / Otsu / Adaptive Threshold
↓
Binary Preview + Foreground Ratio
↓
Threshold Sweep
↓
Result CSV + Intermediate Image Save

```

필수 기능

- 원본 Image 불변 유지
- Threshold Mode와 Polarity Recipe 저장
- ROI 밖 Pixel이 통계에 포함되지 않도록 처리
- 사용 Threshold, Foreground Ratio와 처리 시간 기록
- 정상/빈 ROI/잘못된 Type Unit Test
- Frame별 Normalize 사용 여부와 Mapping 값 기록

12. Chapter 핵심 요약

- ● Histogram은 Gray 분포를 보여주지만 위치 정보는 없다.
- ● Threshold는 Gray Image를 Binary Foreground/Background로 분리한다.
- ● Binary 255는 Object 자체가 아니라 선택된 Foreground다.
- ● Manual, Otsu, Adaptive 방식은 분포와 조명 조건에 따라 선택한다.
- ● Align된 ROI에서 Histogram과 Threshold를 적용해야 한다.
- ● 여러 Sample의 분포로 Threshold Margin을 검증한다.
- ● 8/12/16-bit Threshold 범위와 Pixel Format을 구분한다.
- ● Frame별 Min-Max Normalize는 Gray의 절대 의미를 바꾼다.
- ● Foreground Ratio는 Sanity Check이며 Shape/위치 검증을 대체하지 않는다.
- ● Threshold 실패가 조명 문제라면 Parameter보다 광학 조건을 먼저 개선한다.

9일차 권장 학습 순서

- ☐ 40분: Gray/Histogram/Brightness/Contrast 정리
- ☐ 50분: Manual/Otsu/Adaptive 비교
- ☐ 40분: ROI, Polarity, Bit Depth 주의사항
- ☐ 60~90분: C++ Threshold Pipeline 구현
- ☐ 60분: Threshold Sweep 및 Drift 실습
- ☐ 30분: 면접 Q1~Q6 30초 답변 연습

학습 완료 체크

- ☐ ROI Histogram을 읽고 두 분포와 Margin을 설명한다.
- ☐ Manual/Otsu/Adaptive의 적용 조건을 구분한다.
- ☐ Foreground Polarity를 명확히 정의한다.
- ☐ 8/12-bit Threshold를 올바른 범위로 계산한다.
- ☐ Normalize가 검사 결과를 흔들 수 있는 이유를 설명한다.
- ☐ Threshold Pipeline과 Sweep Tool을 설계하거나 구현했다.

다음 학습 예고

다음 영상처리 Chapter에서는 Mean/Gaussian/Median/Bilateral Filter, Gradient, Sobel, Canny와 Edge Position 측정을 연결한다.

10일차. 영상처리 기초 2: Filtering · Gradient · Edge

Phase 7-2 · 예상 학습 시간: 10~14시간 · 난이도: 중급 · 중요도: ● 반드시 알아야 함

1. 이 Chapter에서 배우는 것

핵심 개념

- ● Filter를 사용하는 이유와 Kernel/Convolution의 의미
- ● Mean, Gaussian, Median, Bilateral Filter의 차이
- ● Noise 제거와 Detail/Edge 보존의 Trade-off
- ● Gradient, Magnitude, Direction
- ● Sobel과 Canny Edge Detection
- ● Edge Polarity와 Search Direction
- ● Edge Position으로 Width/Distance 측정
- ● Border 처리와 ROI 경계 Artifact
- ● Edge Spread, Sub-pixel 위치와 측정 반복성
- ● Filter/Edge Parameter 검증 방법

실무 목표

Noise 종류와 검사 Feature 크기를 기준으로 Filter를 선택하고, Sobel/Canny 결과를 단순히 보기 좋은 Edge Image가 아니라 위치·폭·방향·연결성 측정에 사용한다. Filter가 작은 Defect를 제거하거나 Edge 위치를 이동시킬 수 있음을 이해하고 실제 Sample에서 Bias와 Repeatability를 검증한다.

2. 선수 지식

- 9일차의 Gray, Histogram, ROI, Binary
- Pixel 좌표와 `CV_8UC1`
- 평균, 중앙값, 표준편차
- 미분은 "위치에 따른 값의 변화량"이라는 직관

2.1 오늘의 Pipeline 위치

```

TEXT
Gray ROI
  ↓
Noise/Texture 확인
  ↓
Filter 선택
  ↓
Gradient/Sobel/Canny
  ↓
Edge 후보
  ↓
방향·Polarity·연결성 Filtering
  ↓
Edge Position / Width / Distance / Angle
  
```

Filter와 Edge Detection은 목적 없이 연속 적용하는 고정 절차가 아니다. 원본 Image와 검사 Feature에 따라 필요한 단계만 선택한다.

3. 핵심 개념

3.1 Filtering을 왜 사용하는가?

Filtering의 목적:

- Random Noise 완화
- 작은 밝기 Spike 제거
- Background의 완만한 변화 추정
- Edge/Line 같은 특정 특징 강조
- Downstream Threshold와 Edge 결과 안정화

그러나 모든 Smoothing Filter는 어느 정도 Detail을 잃는다.

TEXT

Noise 감소 ↑

↕ Trade-off

작은 Defect/Sharp Edge 보존 ↓

Kernel 크기를 결함보다 크게 잡으면 검사하려던 결함 자체를 지울 수 있다.

3.2 Kernel과 Convolution

Kernel은 주변 Pixel을 어떤 가중치로 결합할지 정의한 작은 행렬이다.

3×3 Mean Kernel:

TEXT

$$1/9 \times \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

각 출력 Pixel은 주변 3×3 Pixel 평균이 된다. Convolution은 Kernel을 Image 위로 이동하며 이 계산을 반복하는 과정이다.

확인할 Parameter:

- Kernel Size: 3×3, 5×5, 7×7...
- Weight/Shape
- Border 처리
- 입력/출력 Depth
- ROI와 Feature 크기의 관계

3.3 Mean Filter

주변 Pixel을 동일 가중치로 평균한다.

장점:

- 단순하고 빠름
- Random Noise 완화
- Background 평균화

단점:

- Edge와 작은 Defect를 쉽게 Blur
- Salt-and-Pepper Noise에 강하지 않음
- 큰 Kernel에서 Detail 손실

검사 적용:

- 完만한 Background 추정
- 정밀 Edge 측정 전보다 대략적 Segmentation 전처리
- Difference Image의 작은 Random Noise 완화

3.4 Gaussian Filter

중심 Pixel에 큰 가중치, 멀수록 작은 가중치를 사용한다.

TEXT

근사 3×3 Gaussian Kernel

$$1/16 \times \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Parameter:

- Kernel Size
- **sigma** : Gaussian 분포 폭

장점:

- Mean보다 자연스러운 Low-pass
- Gaussian 계열 Random Noise 완화
- Canny 전처리의 일반적인 선택

단점:

- Edge와 작은 Feature가 여전히 Blur
- sigma/Kernel이 커지면 Edge Spread 증가

3.5 Median Filter

주변 Pixel을 정렬한 뒤 중앙값으로 대체하는 비선형 Filter다.

FORMULA · CALCULATION

[10, 11, 10,
12,255, 11,
10, 12, 11]

정렬 중앙값 = 11

중앙의 255 Spike가 제거된다.

장점:

- Salt-and-Pepper/Impulse Noise에 강함
- Mean보다 일부 Edge를 잘 보존

단점:

- 작은 Point/Line Defect도 Noise로 보고 제거할 수 있음
- 큰 Kernel에서 형상 변형
- Gaussian Noise에는 항상 최선이 아님

Particle 검사가 목표라면 Median이 Particle을 지우지 않는지 반드시 확인한다.

3.6 Bilateral Filter

공간적으로 가까운 Pixel뿐 아니라 밝기가 비슷한 Pixel에 더 큰 가중치를 주는 Edge-preserving Filter다.

- `sigmaSpace` : 공간 거리 영향
- `sigmaColor` : Gray 차이 허용 범위

장점:

- 영역 내부 Noise를 줄이면서 큰 Edge 보존

단점:

- Mean/Gaussian보다 계산량이 큼
- Parameter 의미와 성능 최적화가 복잡
- 약한 Defect Contrast가 Background와 섞일 수 있음

실시간 검사에서는 처리 시간과 실제 개선량을 측정하고 사용한다.

3.7 Filter 선택표

상황	1차 후보	이유	주요 위험
Gaussian 계열 Random Noise	Gaussian	주파수/공간적으로 안정적 Smoothing	Edge Spread
Salt-and-Pepper Spike	Median	Outlier 제거	작은 Particle 제거
단순 Background 평균화	Mean/Gaussian	빠르고 해석 쉬움	Detail 손실
큰 Edge 보존하며 영역 Noise 감소	Bilateral	밝기 차이 보존	느림, 약한 Defect 손실
매우 작은 Scratch/Particle	No Filter 또는 작은 Gaussian	Feature 보존 우선	Noise 증가

3.8 Gradient

Gradient는 x/y 방향 Gray 변화량이다.

FORMULA · CALCULATION

$$G_x = x \text{ 방향 미분}$$

$$G_y = y \text{ 방향 미분}$$

$$\text{Magnitude} = \sqrt{G_x^2 + G_y^2}$$

$$\text{Direction} = \text{atan2}(G_y, G_x)$$

- 수직 Edge: x 방향 변화가 크므로 $|G_x|$ 가 큼
- 수평 Edge: y 방향 변화가 크므로 $|G_y|$ 가 큼

Gradient 부호는 밝기 변화 방향을 나타낸다.

TEXT

Dark → Bright: Positive Edge

Bright → Dark: Negative Edge

부호를 활용하면 원하는 방향의 Edge만 선택할 수 있다.

3.9 Sobel Operator

대표적인 3×3 x 방향 Sobel Kernel:

TEXT

```
[-1 0 +1
 -2 0 +2
 -1 0 +1]
```

y 방향:

TEXT

```
[-1 -2 -1
 0 0 0
 +1 +2 +1]
```

Sobel은 미분과 약한 Smoothing을 결합한다. 출력은 8-bit 범위를 넘거나 음수가 될 수 있으므로 **CV_16S** 또는 **CV_32F** 로 계산해야 한다.

[!WARNING]

Sobel 결과를 바로 **CV_8U** 에 저장하면 Negative Gradient가 사라지고 큰 값이 Saturation되어 방향과 크기 정보를 잃을 수 있다.

3.10 Canny Edge Detection

Canny의 주요 단계:

TEXT

Gaussian Smoothing



Gradient 계산



Non-Maximum Suppression



Double Threshold



Hysteresis Tracking

- High Threshold 이상: Strong Edge
- Low~High 사이: Strong Edge와 연결되면 유지
- Low 미만: 제거

장점:

- 비교적 얇고 연결된 Binary Edge
- 후속 Contour/Line 검출에 편리

단점:

- Low/High Threshold와 Blur Parameter에 민감
- Texture가 많으면 Edge가 과다 발생
- Gradient 크기와 Polarity 정보가 Binary 결과에서 사라짐

치수 측정에서는 Canny Pixel을 그대로 사용하는 것보다 1D Profile/Gradient Peak와 Sub-pixel Fit이 더 적합할 수 있다.

3.11 Edge Detection과 Edge Measurement

- **Detection:** Edge가 있는 Pixel 후보를 찾음
- **Localization:** Edge 위치를 결정
- **Measurement:** 두 Edge 간 거리/각도를 실제 단위로 계산

FORMULA · CALCULATION

Left Edge $x_1 = 120.3$ pixel

Right Edge $x_2 = 320.8$ pixel

Width = $(x_2 - x_1) \times \text{scale}$

정밀 Measurement에는 다음을 확인한다.

- Search Direction
- Edge Polarity
- Gradient Peak 선택
- 여러 Edge 중 올바른 Edge ID
- Sub-pixel 방법
- Calibration Scale/Distortion
- Filter에 의한 Bias

3.12 1D Edge Profile

2D Image에서 측정 방향을 따라 여러 Scan Line을 평균하면 Noise를 줄이고 Edge Profile을 얻을 수 있다.

FORMULA · CALCULATION

ROI 여러 행 평균

↓

1D Gray Profile

↓ derivative

1D Gradient

↓

Peak Position = Edge

평균할 행 수가 많으면 Noise는 줄지만 기울어진 Edge나 Curved Edge가 퍼질 수 있다. Align 후 Edge 방향과 Profile 방향을 맞춘다.

3.13 Sub-pixel Edge

Edge 주변 Gradient Peak를 Parabola/Gaussian/Line Model 등으로 Fit하여 Pixel 사이 위치를 추정할 수 있다.

Sub-pixel은 실제 Optical Resolution을 초월하는 새 Detail이 아니다. 충분한 SNR과 안정된 Edge Shape에서 위치 반복성을 개선하는 추정 방법이다.

다음이 바뀌면 Bias가 달라질 수 있다.

- Focus/Blur
- Threshold/Gradient Kernel
- 조명과 Edge Polarity
- Pixel Phase
- Lens Distortion

3.14 Border 처리

Kernel이 Image/ROI 경계를 벗어나면 OpenCV는 Border 값을 만들어 계산한다.

대표 방식:

- Replicate
- Reflect
- Constant

ROI를 잘라 Filter하면 ROI 밖 문맥이 사라져 경계에 Artifact가 생길 수 있다. 필요하다면 ROI를 Margin만큼 확장해 처리한 뒤 원래 ROI로 Crop한다.

4. 그림으로 이해하기

4.1 Noise와 Filter

TEXT

```
Original:  ___/-----\___  + random spikes
Gaussian:  __/~~~~~\__      noise↓ edge spread↑
Median:    ___/-----\___      impulse removed
```

4.2 Edge와 Gradient

TEXT

```
Gray Profile:    20  20  25  80 180 220 220
Gradient:        0   5  55 100 40   0
                  ▲
                Edge 후보 위치
```

4.3 실제 측정 흐름

TEXT

```
Align된 Measurement ROI
    ↓
방향에 수직인 Gray Profile
    ↓
Filter + Gradient
    ↓
Polarity/Strength로 Left·Right Edge 선택
    ↓
Sub-pixel Position
    ↓
Pixel Distance × Calibration
```

5. 실제 검사 장비에서 어디에 사용하는가?

Back Light Width 측정

고Contrast 외곽에서 좌→우 Profile을 만들고 Negative/Positive Edge Pair를 선택해 Width를 구한다. Canny 전체 Contour 보다 측정 방향과 Edge Pair를 명시하는 방식이 해석하기 쉽다.

Scratch 검사

조명 방향과 Scratch 방향에 맞춰 Gradient/Line Response를 사용한다. Gaussian Blur가 너무 크면 얇은 Scratch가 사라지고, 너무 작으면 Texture Edge가 과다 검출된다.

Gap 검사

두 부품 Edge를 각각 찾고 Calibration된 거리로 Gap을 측정한다. 잘못된 내부 Texture Edge를 선택하지 않도록 Search Range, Polarity, Expected Position을 사용한다.

Particle 전처리

Impulse Noise는 Median으로 줄일 수 있지만 실제 Particle도 함께 제거될 수 있다. Raw Image와 Filtered Image의 Blob Area/Count 차이를 검증한다.

Focus Monitoring

Golden Edge의 Gradient Peak, Edge Spread Width, Laplacian Variance 등을 추적해 Focus 변화를 감시할 수 있다. 제품 Pattern 변화와 Focus 변화를 구분할 Reference가 필요하다.

6. 숫자로 이해하기

예제 1: Mean Filter와 Impulse

3×3 영역에서 중심만 255이고 나머지가 0이라면:

FORMULA · CALCULATION

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 255 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

$$\text{Mean} = 255 / 9 \approx 28.33$$

한 Pixel Spike가 주변 3×3 영역으로 낮게 퍼진다. Median은 9개 중 중앙값 0을 선택해 Spike를 제거한다.

예제 2: Sobel Gx

3×3 Patch가 왼쪽 0, 오른쪽 100이라고 하자.

FORMULA · CALCULATION

Image: Sobel Gx:

$$\begin{bmatrix} 0 & 0 & 100 \\ 0 & 0 & 100 \\ 0 & 0 & 100 \end{bmatrix} \times \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} \rightarrow 100+200+100 = 400$$

출력이 255보다 크므로 8-bit에 바로 저장하면 Magnitude를 잃는다. CV_32F 에서 400으로 유지한 뒤 Display/Threshold 목적에 맞게 변환한다.

예제 3: Edge Width Measurement

FORMULA · CALCULATION

Left Edge = 120.3 pixel
 Right Edge = 320.8 pixel
 Scale = 5 μ m/pixel

 Pixel Width = 320.8-120.3 = 200.5 pixel
 Object Width = 200.5 $\times$ 5 = 1002.5 μ m = 1.0025 mm

Scale이 단일값으로 유효하다는 전제이며 정밀 측정에는 Calibration Transform을 사용한다.

예제 4: Profile 평균과 Noise

서로 독립이고 같은 표준편차 $\sigma=12$ 인 Scan Line 16개를 평균하면 이상적으로 평균 Noise 표준편차는:

FORMULA · CALCULATION

$\sigma_{\text{mean}} = 12 / \text{sqrt}(16) = 3$

실제 Image Noise가 공간적으로 상관되어 있거나 Edge가 기울어져 있으면 이론만큼 줄지 않는다.

예제 5: Canny Threshold

Gradient Magnitude가 다음과 같고 Low=40, High=100이라면:

TEXT

30 → 제거
 60 → Weak Edge, Strong Edge와 연결될 때 유지
 130 → Strong Edge

Low/High 비율에 보편적인 정답은 없다. 실제 Edge/Noise Gradient 분포로 정한다.

7. C++ 구현

Filter와 Edge Pipeline

CPP · 1/3

```
#include <opencv2/opencv.hpp>

#include <cmath>
#include <stdexcept>

enum class FilterType {
    None,
    Mean,
    Gaussian,
    Median,
    Bilateral
};

enum class EdgeType {
    Sobel,
    Canny
};

struct FilterRecipe final {
    FilterType type{FilterType::Gaussian};
    int kernelSize{3};
    double gaussianSigma{1.0};
    double bilateralSigmaColor{25.0};
    double bilateralSigmaSpace{3.0};
};

struct EdgeRecipe final {
    EdgeType type{EdgeType::Canny};
    double sobelMagnitudeThreshold{50.0};
    double cannyLowThreshold{40.0};
    double cannyHighThreshold{100.0};
};

struct EdgeResult final {
    cv::Mat filtered;
    cv::Mat gradientX;
    cv::Mat gradientY;
    cv::Mat magnitude;
    cv::Mat binaryEdges;
};
```

CPP · 2/3

```

[[nodiscard]] cv::Mat ApplyFilter(
    const cv::Mat& gray,
    const FilterRecipe& recipe)
{
    if (gray.empty() || gray.type() != CV_8UC1) {
        throw std::invalid_argument{"CV_8UC1 image is required"};
    }
    if (recipe.kernelSize <= 0 || recipe.kernelSize % 2 == 0) {
        throw std::invalid_argument{"Kernel size must be positive and odd"};
    }

    cv::Mat output;
    if (recipe.type == FilterType::None || recipe.kernelSize == 1) {
        output = gray.clone();
        return output;
    }

    switch (recipe.type) {
    case FilterType::None:
        output = gray.clone();
        break;
    case FilterType::Mean:
        cv::blur(gray, output, cv::Size{recipe.kernelSize, recipe.kernelSize});
        break;
    case FilterType::Gaussian:
        if (!std::isfinite(recipe.gaussianSigma) ||
            recipe.gaussianSigma <= 0.0) {
            throw std::invalid_argument{"Gaussian sigma must be positive"};
        }
        cv::GaussianBlur(gray, output,
                        cv::Size{recipe.kernelSize, recipe.kernelSize},
                        recipe.gaussianSigma);
        break;
    case FilterType::Median:
        cv::medianBlur(gray, output, recipe.kernelSize);
        break;
    case FilterType::Bilateral:
        if (!std::isfinite(recipe.bilateralSigmaColor) ||
            !std::isfinite(recipe.bilateralSigmaSpace) ||
            recipe.bilateralSigmaColor <= 0.0 ||
            recipe.bilateralSigmaSpace <= 0.0) {
            throw std::invalid_argument{"Bilateral sigma must be positive"};
        }
    }
}

```

CPP · 3/3

```

        cv::bilateralFilter(gray, output, recipe.kernelSize,
                           recipe.bilateralSigmaColor,
                           recipe.bilateralSigmaSpace);

        break;
    }
    return output;
}

[[nodiscard]] EdgeResult DetectEdges(
    const cv::Mat& gray,
    const FilterRecipe& filterRecipe,
    const EdgeRecipe& edgeRecipe)
{
    EdgeResult result;
    result.filtered = ApplyFilter(gray, filterRecipe);

    cv::Sobel(result.filtered, result.gradientX, CV_32F, 1, 0, 3);
    cv::Sobel(result.filtered, result.gradientY, CV_32F, 0, 1, 3);
    cv::magnitude(result.gradientX, result.gradientY, result.magnitude);

    switch (edgeRecipe.type) {
    case EdgeType::Sobel:
        if (!std::isfinite(edgeRecipe.sobelMagnitudeThreshold) ||
            edgeRecipe.sobelMagnitudeThreshold < 0.0) {
            throw std::invalid_argument{"Invalid Sobel threshold"};
        }
        cv::compare(result.magnitude,
                    edgeRecipe.sobelMagnitudeThreshold,
                    result.binaryEdges, cv::CMP_GE);

        break;

    case EdgeType::Canny:
        if (!std::isfinite(edgeRecipe.cannyLowThreshold) ||
            !std::isfinite(edgeRecipe.cannyHighThreshold) ||
            edgeRecipe.cannyLowThreshold < 0.0 ||
            edgeRecipe.cannyLowThreshold >= edgeRecipe.cannyHighThreshold) {
            throw std::invalid_argument{"Invalid Canny thresholds"};
        }
        cv::Canny(result.filtered, result.binaryEdges,
                  edgeRecipe.cannyLowThreshold,
                  edgeRecipe.cannyHighThreshold,
                  3, true);

        break;
    }
    return result;
}

```

Unit Test

CPP

```
#include <cassert>

void TestMeanPreservesConstantImage()
{
    const cv::Mat image(20, 20, CV_8UC1, cv::Scalar(123));
    FilterRecipe recipe;
    recipe.type = FilterType::Mean;
    recipe.kernelSize = 5;

    const cv::Mat filtered = ApplyFilter(image, recipe);
    assert(cv::countNonZero(filtered != image) == 0);
}

void TestMedianRemovesSingleImpulse()
{
    cv::Mat image = cv::Mat::zeros(5, 5, CV_8UC1);
    image.at<unsigned char>(2, 2) = 255;

    FilterRecipe recipe;
    recipe.type = FilterType::Median;
    recipe.kernelSize = 3;

    const cv::Mat filtered = ApplyFilter(image, recipe);
    assert(filtered.at<unsigned char>(2, 2) == 0);
}

void TestCannyFindsVerticalStep()
{
    cv::Mat image = cv::Mat::zeros(100, 100, CV_8UC1);
    image(cv::Rect{50, 0, 50, 100}).setTo(255);

    FilterRecipe filter;
    filter.type = FilterType::Gaussian;
    filter.kernelSize = 3;
    EdgeRecipe edge;
    edge.type = EdgeType::Canny;
    edge.cannyLowThreshold = 40.0;
    edge.cannyHighThreshold = 100.0;

    const auto result = DetectEdges(image, filter, edge);
    assert(cv::countNonZero(result.binaryEdges) > 0);
    assert(result.gradientX.type() == CV_32FC1);
}
```

코드에서 봐야 할 점

1. Filter 선택과 Edge 선택을 독립 Recipe로 분리했다.
2. 입력 Image를 수정하지 않고 Filtered Image를 별도 생성한다.
3. Sobel Gx/Gy와 Magnitude를 **CV_32F** 로 유지한다.
4. Canny를 사용하더라도 Gradient Image를 Result에 남겨 원인 분석이 가능하다.
5. Kernel Size, Sigma와 Threshold의 범위를 사용 전에 검증한다.
6. 실제 Measurement에서는 Binary Edge 전체가 아니라 방향/Polarity/예상 위치로 올바른 Edge를 선택한다.

8. 실무에서 발생하는 문제

문제 1: 큰 Gaussian Kernel로 작은 Defect가 사라짐

- 대응: Feature Width 대비 Kernel/Sigma Sweep, Raw/Filtered 검출률 비교

문제 2: Median이 실제 Particle 제거

- 대응: Noise와 최소 Particle 크기 분포 비교, Filter 전후 Blob Count 저장

문제 3: 8-bit Sobel로 Negative Edge 손실

- 대응: Signed/Float Depth에서 Gx/Gy 유지, Display 변환 분리

문제 4: Canny가 Texture를 모두 Edge로 검출

- 대응: 조명 개선, ROI, Gradient 방향/크기, 연결 길이와 형상 조건

문제 5: ROI 경계가 강한 Edge가 됨

- 대응: 원본 문맥 포함 Margin ROI에서 Filter/Edge 후 내부 ROI만 사용

문제 6: 여러 Edge 중 잘못된 Pair 선택

- 대응: Expected Position/Width, Polarity, Search Direction, Score와 Recipe Range

문제 7: Focus 변화로 측정 Bias

- 대응: Edge Spread/Gradient Monitoring, Calibration Target 반복 측정, Focus 관리

문제 8: Bilateral로 Cycle Time 초과

- 대응: ROI 축소, Parameter/해상도 최적화, 단순 Filter 대비 이득 계측

9. 흔한 오해

Noise와 함께 검사 Detail도 손실되며 단계마다 Bias가 추가될 수 있다.

1. “Filter를 많이 적용할수록 Image 품질이 좋다.”

Kernel보다 작은 Point Feature는 제거될 수 있다.

2. “Median은 Edge를 보존하므로 작은 Particle도 보존한다.”

Parameter와 Texture에서 생성된 후보이며 올바른 Edge ID를 선택해야 한다.

3. “Canny Edge가 곧 실제 Object 경계다.”

Magnitude는 절댓값 정보이고 Gx/Gy 또는 `atan2` 가 방향을 제공한다.

4. “Sobel Magnitude만 보면 Edge 방향도 안다.”

안정된 Edge 위치를 추정하는 것이며 새로운 Optical Detail을 생성하지 않는다.

5. “Sub-pixel이면 Pixel Size보다 작은 Defect를 볼 수 있다.”

Edge Spread와 Feature 손실, 처리 시간이 증가한다.

6. “큰 Kernel은 Noise가 더 줄어 항상 유리하다.”

Distortion, Focus, Edge Model, Calibration과 반복성 오차를 포함해야 한다.

7. “Edge Pixel 거리×Scale이면 정확한 치수다.”

10. 면접에서 나올 수 있는 질문

Q1. Mean, Gaussian, Median Filter의 차이는 무엇인가요?

초보자가 이해할 수 있는 답변 Mean은 주변 평균, Gaussian은 중심에 더 큰 가중치를 둔 평균, Median은 정렬한 값의 중앙 값을 사용합니다.

실무자 답변 Mean/Gaussian은 Linear Low-pass로 Random Noise를 줄이지만 Edge를 Blur한다. Median은 비선형 Order Filter로 Impulse Noise에 강하지만 작은 Particle/Line을 제거할 수 있다. Noise와 최소 Feature 크기로 선택한다.

면접용 30초 답변 “Mean은 동일 가중 평균, Gaussian은 거리 기반 가중 평균, Median은 주변 중앙값입니다. Gaussian은 Random Noise, Median은 Salt-and-Pepper에 주로 검토하며, 어떤 Filter든 최소 Defect와 Edge Bias를 실제 Sample로 확인합니다.”

Q2. Bilateral Filter는 언제 사용하나요?

초보자가 이해할 수 있는 답변 영역 내부 Noise는 줄이면서 밝기 차이가 큰 Edge는 최대한 유지하고 싶을 때 사용합니다.

실무자 답변 공간 거리와 Intensity 차이를 함께 가중해 큰 Edge를 보존한다. sigmaColor/Space가 약한 결함 Contrast를 섞지 않는지와 높은 계산 비용이 Cycle Time에 가치가 있는지 검증한다.

면접용 30초 답변 “Bilateral은 가까우면서 밝기도 비슷한 Pixel을 평균해 큰 Edge를 보존합니다. 다만 느리고 약한 Defect도 Background로 평활화할 수 있어 Gaussian 대비 검출률 개선과 처리 시간을 측정한 뒤 사용합니다.”

Q3. Sobel Gx와 Gy는 무엇을 의미하나요?

초보자가 이해할 수 있는 답변 Gx는 x 방향 밝기 변화, Gy는 y 방향 밝기 변화입니다. 수직 Edge는 Gx가 크고 수평 Edge는 Gy가 큼니다.

실무자 답변 Sobel은 미분과 직교 방향 Smoothing을 결합한다. Signed Gx/Gy로 Polarity와 Gradient 방향을 얻고 Magnitude로 강도를 계산한다. 출력 범위를 위해 16-bit Signed 또는 Float를 사용한다.

면접용 30초 답변 “Gx는 좌우 밝기 변화라 수직 Edge에, Gy는 상하 변화라 수평 Edge에 강하게 응답합니다. $\sqrt{Gx^2 + Gy^2}$ 로 강도, $\text{atan2}(Gy, Gx)$ 로 방향을 얻으며 부호는 Dark-to-Bright 같은 Polarity를 나타냅니다.”

Q4. Canny의 Low/High Threshold는 어떤 역할인가요?

초보자가 이해할 수 있는 답변 High보다 강하면 확실한 Edge, Low와 High 사이는 강한 Edge와 연결될 때만 유지합니다.

실무자 답변 Double Threshold/Hysteresis가 약한 Edge 중 구조적으로 Strong Edge와 연결된 것만 보존한다. Gradient/Noise 분포, Filter와 조명 조건으로 정하고 Result의 연결성/False Edge를 검증한다.

면접용 30초 답변 “High 이상은 Strong Edge, Low~High는 Strong Edge와 연결된 경우만 유지하는 Weak Edge입니다. 고정 비율을 맹신하지 않고 실제 Gradient 분포와 필요한 Edge 연결성, Texture 오검출을 기준으로 정합니다.”

Q5. Edge로 치수를 측정할 때 무엇을 확인해야 하나요?

초보자가 이해할 수 있는 답변 원하는 방향과 밝기 변화의 Edge인지 확인하고 두 Edge 위치 차이에 Calibration Scale을 적용합니다.

실무자 답변 Search ROI/Direction, Polarity, Expected Position, Edge Strength와 Pair ID를 고정한다. Sub-pixel Model, Filter/Focus Bias, Distortion Calibration, Repeatability와 Gauge R&R을 검증한다.

면접용 30초 답변 “먼저 Align된 ROI에서 Search 방향과 Polarity로 올바른 Edge Pair를 선택합니다. Sub-pixel 위치 차이에 Calibration을 적용하고, Focus-Filter-조명 변화에 따른 Bias와 반복 정밀도가 측정 공차를 만족하는지 확인합니다.”

Q6. Filtering이 Edge 위치를 바꿀 수 있나요?

초보자가 이해할 수 있는 답변 대칭적인 이상 Edge에서는 중심이 유지될 수 있지만 실제 비대칭 형상, 주변 Pattern과 경계에서는 위치가 달라질 수 있습니다.

실무자 답변 대칭 Kernel과 고립 Step Edge의 이론적 중심은 유지되지만 PSF, 비대칭 밝기, 인접 Edge, ROI Border, Nonlinear Median/Bilateral과 Threshold 방식이 Bias를 만든다. Golden Gauge로 Filter별 Bias를 계측한다.

면접용 30초 답변 “가능합니다. 이상적 대칭 Step Edge에서는 중심이 유지될 수 있지만 실제 Image는 비대칭 Blur, 주변 Edge와 ROI Border가 있어 Filter가 Gradient Peak나 Threshold 위치를 이동시킬 수 있습니다. 따라서 Filter 전후 Bias와 반복성을 측정합니다.”

11. 실습 문제

실습 1: Filter 비교

동일 Image에 None/Mean/Gaussian/Median/Bilateral을 적용하고 다음을 비교한다.

- Background 표준편차
- Edge Gradient Peak
- Edge Spread Width
- 최소 Defect Contrast
- 처리 시간

실습 2: Kernel Sweep

Gaussian Kernel 1/3/5/7과 sigma를 바꾸며 Scratch/Particle 검출률과 Noise Edge 수를 기록한다. “가장 매끄러운 Image”가 아니라 검사 결과가 가장 안정적인 Parameter를 선택한다.

실습 3: Edge Profile 측정

Back Light 부품 Image에서 여러 Scan Line을 평균해 1D Profile을 만들고 Left/Right Gradient Peak로 Width를 측정한다. Scan Line 수에 따른 Noise와 Edge Bias를 비교한다.

실습 4: Canny Parameter Map

Low/High Threshold 조합을 Sweep하고 다음을 표로 저장한다.

- Edge Pixel Count
- Connected Component 수
- Target Edge 연결률
- False Edge 수
- 처리 시간

Phase 7-2 미니 프로젝트: Edge Measurement Workbench

TEXT

```

Image + Align된 ROI
↓
Filter 선택/Parameter Sweep
↓
Gx/Gy/Magnitude/Canny 시각화
↓
1D Profile + Edge Polarity/Peak
↓
Sub-pixel Left/Right Position
↓
Calibration Width
↓
Bias / Repeatability Report

```

필수 기능

- Raw/Filtered/Gradient/Binary Edge 저장
- Filter와 Edge Recipe Version
- Search Direction, Polarity, Expected Range
- Pixel/실제 단위 측정값 동시 저장
- Filter별 처리 시간과 측정 Bias 비교
- Constant/Impulse/Step Edge Unit Test

12. Chapter 핵심 요약

- ● Filter는 Noise와 함께 Detail도 바꾸므로 목적과 Feature 크기로 선택한다.
- ● Mean/Gaussian은 평균 기반, Median은 Outlier 기반 비선형 Filter다.
- ● Bilateral은 큰 Edge를 보존하지만 느리고 약한 결함을 지울 수 있다.
- ● Gradient의 Gx/Gy는 변화 방향과 Polarity를 제공한다.
- ● Sobel 결과는 Signed/Float Depth로 유지한다.
- ● Canny는 얇은 Binary Edge 후보를 만들지만 Measurement 자체는 아니다.
- ● Edge 측정에는 방향, Polarity, 위치 범위와 Calibration이 필요하다.
- ● Sub-pixel은 위치 추정 반복성을 개선하지만 Optical Detail을 새로 만들지 않는다.
- ● ROI Border와 Filter가 Edge Bias를 만들 수 있다.
- ● Parameter는 검출률, Bias, 반복성, 처리 시간으로 검증한다.

10일차 권장 학습 순서

- ☐ 45분: Mean/Gaussian/Median/Bilateral 비교
- ☐ 45분: Gradient, Gx/Gy, Magnitude/Direction 계산
- ☐ 40분: Sobel/Canny Pipeline 정리
- ☐ 50분: Edge Detection과 Measurement 차이
- ☐ 60~90분: C++ Filter/Edge Pipeline 구현
- ☐ 60분: Filter-Canny Parameter Sweep
- ☐ 30분: 면접 Q1~Q6 30초 답변 연습

학습 완료 체크

- ☐ Noise와 Feature 크기로 Filter 후보를 선택한다.
- ☐ Mean/Gaussian/Median/Bilateral의 Trade-off를 설명한다.
- ☐ Sobel Gx/Gy와 Edge 방향/Polarity를 설명한다.
- ☐ Canny의 Double Threshold/Hysteresis를 설명한다.
- ☐ Edge Pair로 실제 Width를 계산한다.
- ☐ Filter가 Measurement Bias를 만들 수 있음을 검증한다.
- ☐ Edge Measurement Workbench를 설계하거나 구현했다.

다음 학습 예고

다음 영상처리 Chapter에서는 Erosion, Dilation, Opening, Closing과 Connected Component/Blob을 연결해 Binary Object의 Area, Bounding Box, Centroid, Circularity를 측정한다.

11일차. 영상처리 기초 3: Morphology · Connected Component · Blob

Phase 7-3 · 예상 학습 시간: 10~14시간 · 난이도: 중급 · 중요도: ● 반드시 알아야 함

1. 이 Chapter에서 배우는 것

핵심 개념

- ● Binary Foreground와 Structuring Element(Kernel)
- ● Erosion, Dilation, Opening, Closing
- ● Morphology가 형상과 측정값에 주는 영향
- ● 4/8 Connectivity와 Connected Component Labeling
- ● Blob의 Area, Bounding Box, Centroid, Width/Height

- ● Perimeter, Circularity, Aspect Ratio
- ● Pixel 면적을 실제 단위로 변환하는 방법
- ● Border Blob, Hole, 붙은 Blob과 분리 문제
- ● Blob 조건을 이용한 Presence/Particle/결함 검사

실무 목표

Threshold 결과에 Morphology를 무조건 적용하지 않고 제거할 Noise와 보존할 최소 Defect 크기를 기준으로 Kernel을 선택한다. Connected Component/Blob 특징을 측정해 후보를 Filtering하고, Morphology 전후 측정 Bias와 Calibration 단위를 검증한다.

2. 선수 지식

- 9일차의 Binary, Polarity, ROI
- 10일차의 Kernel과 Border
- Pixel 좌표, $\mu\text{m}/\text{pixel}$, 면적 단위

이 Chapter에서는 255=검사할 Foreground, 0=Background 로 통일한다. Polarity가 반대면 먼저 Binary를 반전한다.

3. 핵심 개념

3.1 Structuring Element

Morphology는 Kernel 안에서 Foreground의 형태를 비교·변형한다.

TEXT		
Rect 3×3	Cross 3×3	Ellipse 3×3
1 1 1	0 1 0	0 1 0
1 1 1	1 1 1	1 1 1
1 1 1	0 1 0	0 1 0

Kernel의 크기와 모양이 처리할 방향과 최소 형상을 결정한다. $5 \mu\text{m}/\text{pixel}$ 에서 3×3 Kernel의 폭은 약 15 μm 이며, 한 번의 Erosion은 평평한 경계를 각 방향 약 1 Pixel(5 μm) 후퇴시킨다.

3.2 Erosion(침식)

Kernel이 Foreground 내부에 완전히 들어갈 때만 중심을 Foreground로 유지한다.

효과:

- 밝은 Foreground 축소
- 작은 밝은 Noise 제거
- 가는 연결부 끊기
- Hole 확대

위험:

- 작은 실제 Defect 소실
- Width/Area 감소
- 얇은 Line 단절

3.3 Dilation(팽창)

Kernel 안에 Foreground가 하나라도 있으면 중심을 Foreground로 만든다.

효과:

- Foreground 확대
- 작은 Hole/Gap 축소
- 가까운 Blob 연결
- 끊긴 Line 보완

위험:

- 서로 다른 결함이 하나로 합쳐짐
- Width/Area 증가
- Background Noise 확대

3.4 Opening

FORMULA · CALCULATION

Opening = Erosion → Dilation

큰 형상의 전체 크기를 대략 복원하면서 Kernel보다 작은 밝은 Foreground를 제거한다.

적용:

- Binary의 작은 밝은 Noise
- 큰 제품 외곽 주변의 작은 돌기 제거

위험:

- 작은 Particle/Scratch 자체가 검사 대상이면 함께 제거
- Corner가 둥글어지고 가는 구조가 끊길 수 있음

3.5 Closing

FORMULA · CALCULATION

Closing = Dilation → Erosion

작은 검은 Hole과 Gap을 메우고 가까운 Foreground를 연결한다.

적용:

- Threshold로 끊긴 문자/외곽 연결
- Blob 내부 작은 Hole 채우기

위험:

- 서로 다른 Particle이 합쳐짐
- 실제 Crack/Gap을 제거

3.6 Morphology 비교

연산	Foreground 효과	대표 사용	측정 위험
Erosion	축소	작은 밝은 Noise/접촉 분리	Area-Width 감소
Dilation	확대	Gap 연결/Hole 축소	Area-Width 증가
Opening	작은 밝은 형상 제거	Speckle 제거	작은 Defect 소실
Closing	작은 어두운 Gap 제거	끊긴 Object 연결	결함/Blob 병합

Morphology 결과를 최종 치수 측정에 사용해야 한다면 Golden Gauge로 Bias를 계측한다. 가능하면 Morphology는 후보 검출에 사용하고 정밀 Edge/치수는 Gray 원본에서 다시 측정한다.

3.7 Connectivity

TEXT

```
1 0
0 1
```

- 4-connectivity: 상하좌우만 연결. 위 두 Pixel은 별개.
- 8-connectivity: 대각선 포함. 위 두 Pixel은 하나.

대각선 접촉을 같은 Object로 볼지 검사 의미에 따라 정한다. OpenCV Connected Components에서는 Connectivity를 명시한다.

3.8 Connected Component Labeling

Binary Foreground를 연결된 그룹으로 분리해 각 그룹에 Label을 붙인다.

TEXT

```
0 0 1 1 0 2
0 0 1 1 0 2
3 0 0 0 0 0
```

각 Label에서 Area, Bounding Box, Centroid를 얻는다. Background는 보통 Label 0이다.

3.9 Blob 특징

- Area: Foreground Pixel 수
- Bounding Box: Blob을 포함하는 최소 축 정렬 사각형
- Centroid: Pixel/면적 중심
- Width/Height: Bounding Box 크기
- Perimeter: Contour 길이
- Aspect Ratio: Width/Height
- Circularity: $4\pi \times \text{Area} / \text{Perimeter}^2$
- Border Touch: ROI/Image 경계 접촉 여부

Circularity는 이상적인 원에서 1에 가깝고 길거나 불규칙하면 작아진다. 작은 Blob은 Pixel Quantization 때문에 값이 불안정하다.

3.10 Blob과 실제 검사 연결

검사	주요 Blob 특징	추가 조건
Presence	Count, Area, Position	예상 ROI
Particle	Area, Peak Gray, Circularity	Dark Field CNR
Missing	Expected Blob 부재	Align/ROI 정상 여부
Stain	Area, Mean Difference, Elongation	Background 보정
Burr	Border 위치, Area, 방향	외곽 기준
Hole	내부 Background Component	Contour Hierarchy
Scratch	길이, Aspect, Orientation	Line/Gradient와 결합

3.11 Pixel 면적을 실제 단위로 변환

X/Y Scale이 각각 s_x , s_y $\mu\text{m}/\text{pixel}$ 이면:

FORMULA · CALCULATION

$$\text{Area}(\mu\text{m}^2) = \text{Pixel Area} \times s_x \times s_y$$

단일 $\mu\text{m}/\text{pixel}$ 을 제공하는 방식은 X/Y Scale이 같고 Perspective/Distortion이 작은 경우의 근사다. 정밀 면적은 Calibration Transform의 위치별 Jacobian 또는 보정 Image를 사용한다.

3.12 Border Blob

ROI 경계에 닿은 Blob은 실제 전체 크기를 알 수 없다.

- 검사 대상이 ROI 밖으로 잘림
- Threshold가 ROI 경계를 Object Edge로 만들
- Align 오차로 Object가 이동

Area/Width 판정 전에 Border Touch를 별도 상태로 처리하거나 Margin ROI에서 검출한다.

3.13 붙은 Blob과 떨어진 Blob

Touching Blob 분리 방법:

- Erosion/Opening
- Distance Transform + Watershed
- Concavity/Contour 분석
- Gray Image의 Local Peak

끊긴 Blob 연결 방법:

- Dilation/Closing
- 방향성 Kernel
- Line/Contour Model

Morphology만으로 물리적으로 다른 Object를 정확히 복원한다고 가정하지 않는다.

4. 그림으로 이해하기

TEXT

Original	Erosion (shrink)	Dilation

Opening: small white dot removed
Closing: small black hole filled

TEXT

```

Binary → Morphology → Labeling → Feature Filter → Measurement
      |           |           |           |
      shape changes components area/position/circularity
  
```

5. 실제 검사 장비에서 어디에 사용하는가?

Particle 검사

Threshold 후 작은 Component를 찾는다. Median/Opening이 실제 최소 Particle까지 제거하지 않는지 확인하고 Area뿐 아니라 Gray Contrast와 위치를 함께 판정한다.

Presence 검사

Expected ROI 안에 지정 Area/Position의 Blob이 정확히 하나 있는지 확인한다. Count가 0이면 Missing, 여러 개면 파손/Noise 가능성을 구분한다.

외곽 Burr 검사

정상 외곽 Mask와 현재 Binary의 차이를 구해 바깥쪽 Blob Area/길이를 측정한다. ROI Border Blob과 정상 위치 오차가 Burr로 판정되지 않도록 Align이 선행되어야 한다.

문자/Pattern 연결

끊긴 Stroke를 방향성 Closing으로 연결할 수 있다. Kernel 방향과 길이가 다른 문자를 합치지 않는지 OCR/Matching 결과로 검증한다.

6. 숫자로 이해하기

예제 1: 5×5 정사각형과 3×3 Kernel

무한 Background 안의 5×5 Foreground 정사각형에 Rect 3×3을 한 번 적용하면 이상적으로:

FORMULA · CALCULATION

Original Area = $5 \times 5 = 25 \text{ pixel}^2$

Erosion Area = $3 \times 3 = 9 \text{ pixel}^2$

Dilation Area = $7 \times 7 = 49 \text{ pixel}^2$

예제 2: 실제 면적 변환

FORMULA · CALCULATION

Blob Area = 800 pixel^2

$s_x = s_y = 5 \text{ } \mu\text{m/pixel}$

Area = $800 \times 5 \times 5 = 20,000 \text{ } \mu\text{m}^2$
 $= 0.020 \text{ mm}^2$

$1 \text{ mm}^2 = 1,000,000 \text{ } \mu\text{m}^2$ 를 사용했다.

예제 3: Circularity

Area 314 pixel^2 , Perimeter 62.8 pixel 인 Blob:

FORMULA · CALCULATION

Circularity = $4\pi \times 314 / 62.8^2 \approx 1.000$

동일 Area에서 Perimeter가 100이면 약 0.395로 불규칙하거나 긴 형상일 가능성이 높다.

예제 4: 회전각 Noise와 Centroid

Blob Pixel 좌표 $(10, 10), (11, 10), (10, 11), (11, 11)$ 의 Centroid:

FORMULA · CALCULATION

$$x_c = (10 + 11 + 10 + 11) / 4 = 10.5$$

$$y_c = (10 + 10 + 11 + 11) / 4 = 10.5$$

Centroid가 소수라고 해서 실제 광학 정밀도가 자동으로 Sub-pixel 수준이 되는 것은 아니다.

7. C++ 구현

CPP · 1/3

```
#include <opencv2/opencv.hpp>

#include <cmath>
#include <stdexcept>
#include <vector>

enum class MorphOperation { None, Erode, Dilate, Open, Close };
enum class KernelShape { Rect, Cross, Ellipse };

struct MorphRecipe final {
    MorphOperation operation{MorphOperation::None};
    KernelShape shape{KernelShape::Ellipse};
    int kernelSize{3};
    int iterations{1};
};

struct BlobFeature final {
    int label{};
    int areaPixels{};
    cv::Rect boundingBox;
    cv::Point2d centroid;
    double perimeterPixels{};
    double circularity{};
    double aspectRatio{};
    bool touchesBorder{};
};

[[nodiscard]] int ToOpenCvShape(const KernelShape shape)
{
    switch (shape) {
        case KernelShape::Rect: return cv::MORPH_RECT;
        case KernelShape::Cross: return cv::MORPH_CROSS;
        case KernelShape::Ellipse: return cv::MORPH_ELLIPSE;
    }
    throw std::invalid_argument{"Unknown kernel shape"};
}

[[nodiscard]] cv::Mat ApplyMorphology(
    const cv::Mat& binary, const MorphRecipe& recipe)
{
    if (binary.empty() || binary.type() != CV_8UC1) {
        throw std::invalid_argument{"CV_8UC1 binary image is required"};
    }
}
```

CPP · 2/3

```

    if (recipe.kernelSize <= 0 || recipe.kernelSize % 2 == 0 ||
        recipe.iterations <= 0) {
        throw std::invalid_argument{"Kernel must be positive/odd; iterations positive"};
    }
    if (recipe.operation == MorphOperation::None) {
        return binary.clone();
    }

    const cv::Mat kernel = cv::getStructuringElement(
        ToOpenCvShape(recipe.shape),
        cv::Size{recipe.kernelSize, recipe.kernelSize});
    cv::Mat output;

    switch (recipe.operation) {
    case MorphOperation::None: output = binary.clone(); break;
    case MorphOperation::Erode:
        cv::erode(binary, output, kernel, cv::Point{-1,-1}, recipe.iterations);
        break;
    case MorphOperation::Dilate:
        cv::dilate(binary, output, kernel, cv::Point{-1,-1}, recipe.iterations);
        break;
    case MorphOperation::Open:
        cv::morphologyEx(binary, output, cv::MORPH_OPEN, kernel,
            cv::Point{-1,-1}, recipe.iterations);
        break;
    case MorphOperation::Close:
        cv::morphologyEx(binary, output, cv::MORPH_CLOSE, kernel,
            cv::Point{-1,-1}, recipe.iterations);
        break;
    }
    return output;
}

[[nodiscard]] std::vector<BlobFeature> AnalyzeBlobs(
    const cv::Mat& binary, const int connectivity = 8)
{
    if (binary.empty() || binary.type() != CV_8UC1) {
        throw std::invalid_argument{"CV_8UC1 binary image is required"};
    }
    if (connectivity != 4 && connectivity != 8) {
        throw std::invalid_argument{"Connectivity must be 4 or 8"};
    }
}

```

CPP · 3/3

```

cv::Mat labels, stats, centroids;
const int count = cv::connectedComponentsWithStats(
    binary, labels, stats, centroids, connectivity, CV_32S);
std::vector<BlobFeature> features;
constexpr double pi = 3.14159265358979323846;

for (int label = 1; label < count; ++label) {
    const cv::Rect box{
        stats.at<int>(label, cv::CC_STAT_LEFT),
        stats.at<int>(label, cv::CC_STAT_TOP),
        stats.at<int>(label, cv::CC_STAT_WIDTH),
        stats.at<int>(label, cv::CC_STAT_HEIGHT)};
    const int area = stats.at<int>(label, cv::CC_STAT_AREA);

    cv::Mat componentMask;
    cv::compare(labels, label, componentMask, cv::CMP_EQ);
    std::vector<std::vector<cv::Point>> contours;
    cv::findContours(componentMask, contours, cv::RETR_EXTERNAL,
        cv::CHAIN_APPROX_NONE);
    double perimeter = 0.0;
    for (const auto& contour : contours) {
        perimeter += cv::arcLength(contour, true);
    }
    const double circularity = perimeter > 0.0
        ? 4.0*pi*static_cast<double>(area)/(perimeter*perimeter)
        : 0.0;
    const bool border = box.x == 0 || box.y == 0 ||
        box.x + box.width == binary.cols ||
        box.y + box.height == binary.rows;

    features.push_back({
        label, area, box,
        {centroids.at<double>(label,0), centroids.at<double>(label,1)},
        perimeter, circularity,
        static_cast<double>(box.width)/static_cast<double>(box.height),
        border});
}
return features;
}

```

Unit Test

C++

```
#include <cassert>

void TestSingleRectangleBlob()
{
    cv::Mat image = cv::Mat::zeros(30, 30, CV_8UC1);
    image(cv::Rect{5, 7, 10, 8}).setTo(255);
    const auto blobs = AnalyzeBlobs(image);
    assert(blobs.size() == 1);
    assert(blobs[0].areaPixels == 80);
    assert(blobs[0].boundingBox == cv::Rect(5, 7, 10, 8));
    assert(!blobs[0].touchesBorder);
}

void TestOpeningRemovesOnePixelNoise()
{
    cv::Mat image = cv::Mat::zeros(30, 30, CV_8UC1);
    image(cv::Rect{10,10,7,7}).setTo(255);
    image.at<unsigned char>(2,2)=255;
    const cv::Mat opened = ApplyMorphology(
        image, {MorphOperation::Open, KernelShape::Rect, 3, 1});
    assert(opened.at<unsigned char>(2,2) == 0);
    assert(cv::countNonZero(opened) == 49);
}
```

코드에서 봐야 할 점

1. Binary Polarity와 Type을 호출 전에 고정한다.
2. Morphology와 Blob Analysis를 별도 단계로 둔다.
3. Label 0 Background를 제외한다.
4. `connectedComponentsWithStats` 의 Pixel Area/Centroid와 Contour Perimeter를 결합한다.
5. Border Touch를 Result에 명시한다.
6. Circularity는 작은 Blob과 Pixel Contour에서 1을 넘거나 흔들릴 수 있으므로 경험적 범위를 검증한다.

8. 실무에서 발생하는 문제

1. **Opening이 최소 Particle을 제거**: Kernel을 물리 단위로 환산하고 Filter 전후 검출률 비교.
2. **Closing이 인접 Defect를 병합**: Blob 간 최소 Gap과 Kernel 크기 검증.
3. **ROI Border Blob의 Area 과소 측정**: Margin ROI와 Border 상태 사용.
4. **4/8 Connectivity 변경으로 Count 변화**: Recipe에 Connectivity 저장.
5. **Threshold 변화가 Blob Area 변화로 확대**: Gray 분포와 Threshold Margin Monitoring.
6. **Circularity로 작은 Particle 오판**: 최소 Area 이상에서만 Shape 조건 적용.
7. **Morphology 결과로 정밀 치수 측정**: Gray 원본 Edge에서 재측정.

9. 흔한 오해

1. “Opening은 Noise만 제거한다.” 실제 작은 Defect도 제거한다.
2. “Closing은 끊긴 Object를 원래대로 복원한다.” 다른 Object까지 합칠 수 있다.
3. “Centroid는 Object의 설계 중심이다.” Binary 밝기/형상 중심일 뿐이다.
4. “Circularity 1이면 반드시 원이다.” Pixel화와 작은 Blob에서 불안정하다.
5. “Blob Area×Scale²이면 항상 실제 면적이다.” X/Y Scale과 왜곡을 고려해야 한다.
6. “Component Count가 맞으면 검사도 정상이다.” 위치·크기·형상·Gray Contrast가 필요하다.
7. “Morphology 반복 횟수를 늘리면 더 깨끗하다.” 형상 Bias도 누적된다.

10. 면접에서 나올 수 있는 질문

Q1. Erosion과 Dilation의 차이는?

초보자 답변: Erosion은 흰 Foreground를 줄이고 Dilation은 늘린다. **실무자 답변:** Structuring Element 포함 관계로 Binary 형상을 변형하며 Area/Width Bias와 연결·분리를 만든다. **30초 답변:** “255를 Foreground로 정의하면 Erosion은 경계를 안 쪽으로, Dilation은 바깥쪽으로 이동시킵니다. Noise 제거·Gap 연결에 쓰지만 측정값이 바뀌므로 Kernel을 최소 Defect와 $\mu\text{m}/\text{pixel}$ 기준으로 정합니다.”

Q2. Opening과 Closing은 언제 사용하는가?

초보자 답변: Opening은 작은 흰 Noise를 제거하고 Closing은 작은 검은 Hole/Gap을 메운다. **실무자 답변:** Opening=Erode→Dilate, Closing=Dilate→Erode이며 Kernel보다 작은 구조를 선택적으로 제거한다. **30초 답변:** “Opening은 작은 Foreground, Closing은 작은 Background Gap 제거에 사용합니다. 실제 Particle/Crack도 지울 수 있어 원본과 결과의 검출률·형상 Bias를 비교합니다.”

Q3. 4-connectivity와 8-connectivity의 차이는?

초보자 답변: 4는 상하좌우, 8은 대각선까지 연결한다. **실무자 답변:** 대각선 접촉 Component의 Label/Count가 달라지므로 물리적 Object 정의에 맞춰 Recipe로 고정한다. **30초 답변:** “대각선 접촉을 같은 Blob으로 볼지 결정하는 차이입니다. Connectivity 변경은 Count와 Area Grouping을 바꾸므로 검사 의미와 Sample로 선택합니다.”

Q4. Blob에서 어떤 특징을 측정하는가?

초보자 답변: 면적, 위치 중심, 사각형 크기와 모양을 측정한다. **실무자 답변:** Area/Bounding Box/Centroid/Perimeter/Circularity/Aspect/Border와 원본 Gray 통계를 결합한다. **30초 답변:** “Blob 후보마다 Area, 위치, 폭·높이, Perimeter와 Circularity를 구하고 Expected ROI와 Spec으로 Filtering합니다. Border Blob과 작은 Blob의 Shape 불안정성을 별도 처리합니다.”

Q5. Morphology 후 치수 측정 시 문제는?

초보자 답변: 형상이 줄거나 늘어 실제 크기와 달라질 수 있다. **실무자 답변:** Kernel/iteration에 따른 systematic bias가 생기며 비대칭/인접 형상에서는 위치도 변한다. **30초 답변:** “Morphology는 검출 후보 정리에 사용하고 정밀 치수는 원본 Gray Edge에서 다시 측정하는 편이 안전합니다. 사용해야 한다면 Gauge로 Bias와 반복성을 보정·검증합니다.”

Q6. Pixel Area를 실제 면적으로 어떻게 바꾸는가?

초보자 답변: Pixel 수에 X와 Y 방향 실제 Pixel 크기를 곱한다. **실무자 답변:** $A=A_{\text{px}} \times s_x \times s_y$ 이며 Perspective/Distortion이 크면 위치별 Jacobian 또는 Rectification이 필요하다. **30초 답변:** “단순 Scale이 유효하면 Pixel Area에 X/Y $\mu\text{m-per-pixel}$ 을 곱합니다. 정밀 면적은 왜곡 보정과 위치별 Calibration을 적용하고 Target 면적으로 검증합니다.”

11. 실습 문제

1. 1~9 Pixel Particle Image에서 Rect/Ellipse 3/5 Kernel Opening 전후 생존율을 표로 만든다.
2. 4/8 Connectivity에 따라 Component Count가 달라지는 Test Image를 설계한다.
3. Blob Area 1200 pixel², X=4.8 μ m/pixel, Y=5.1 μ m/pixel의 실제 면적을 계산한다.
4. 정상/Particle/Scratch Blob의 Area, Circularity, Aspect 분포로 Rule을 설계한다.

Phase 7-3 미니 프로젝트: Blob Inspection Workbench

TEXT

Gray ROI → Threshold → Morphology Sweep → Components
→ Area/Position/Shape/Border Filter → Overlay/CSV → OK/NG

원본·Binary·Morphology Image, Recipe, Component 전체 목록과 탈락 이유를 저장한다.

12. Chapter 핵심 요약

- Morphology는 Binary 형상을 바꾸므로 검출과 측정 Bias를 함께 만든다.
- Opening은 작은 Foreground, Closing은 작은 Background Gap을 제거한다.
- Kernel을 Pixel이 아니라 최소 Feature의 실제 크기와 연결한다.
- Connectivity는 Component Count와 의미를 바꾼다.
- Blob은 Area뿐 아니라 위치·형상·Border·Gray 특징으로 판정한다.
- 정밀 치수는 가능하면 Morphology 전 Gray Image에서 재측정한다.
- Pixel 면적 변환에는 X/Y Scale과 Distortion이 필요하다.
- 모든 후보와 탈락 이유를 Result/Review에 남긴다.

11일차 학습 완료 체크

- ☐ Erosion/Dilation/Opening/Closing을 Foreground 기준으로 설명한다.
- ☐ Kernel 크기를 μ m로 환산한다.
- ☐ 4/8 Connectivity를 선택한다.
- ☐ Blob Area/Box/Centroid/Circularity를 계산한다.
- ☐ Border Blob과 Morphology Bias를 처리한다.
- ☐ Blob Workbench를 설계하거나 구현했다.

다음 학습 예고

12일차에는 Static/Dynamic ROI, Align X/Y/Theta를 적용한 ROI 이동·회전과 좌표 Transform을 학습한다.

16일차 A. 산업용 검사 알고리즘 선택과 측정

Phase 13 · 예상 학습 시간: 14~18시간 · 난이도: 중급 · 중요도: ● 반드시 알아야 함

1. 이 Chapter에서 배우는 것

- Presence/Position/Size/Distance/Angle 검사
- Edge, Blob, Pattern Matching의 선택 기준
- Scratch/Particle/Crack/Missing/Stain/Contamination
- Feature Extraction과 Measurement/Judgement 분리
- Reference/Golden Image 기반 Difference
- 검출률, 오검률, 반복성과 Guard Band
- 전통 Vision과 Deep Learning 적용 경계

실무 목표

결함 이름만으로 Algorithm을 선택하지 않고 Image에서 나타나는 밝기·Edge·형상·Texture 특징으로 분해한다. 측정값과 판정을 분리하고 정상/불량 Variation으로 성능을 검증한다.

2. 선수 지식

- Threshold/Edge/Blob/Matching
- Align/ROI/Calibration
- 조명과 CNR

TEXT

Physical Defect → Optical Contrast → Image Feature → Algorithm → Measurement → Spec

3. 핵심 개념

3.1 검사 문제를 Feature로 번역

TEXT

“Scratch를 찾아라”

- 배경보다 밝거나 어두운가?
- 길고 가는 Line/Edge인가?
- 방향/폭/길이 범위는?
- 표면 Texture와 어떻게 다른가?

3.2 기본 검사 선택표

검사	주요 Feature	대표 Algorithm	주요 Risk
Presence/Absence	Area/Pattern 유무	Blob, Matching	위치 편차/가림
Position	Mark/Edge Center	Matching, Edge, Blob Centroid	Calibration
Width/Height	두 반대 Edge	1D Profile/Sobel/Sub-pixel	Focus/Polarity
Area	Binary Region	Threshold+Component	Threshold Bias
Distance	Point/Line 간 거리	Edge/Geometry+Calibration	Distortion
Angle	Line/Mark Vector	Line Fit, Matching	Baseline
Diameter	Circle Edge	Circle/Ellipse Fit	Perspective
Roundness	Radius Variation	Contour/Circle Residual	Pixelization
Edge Position	Gradient Peak	Profile/Sobel	Blur/Bias

3.3 Defect 검사 선택표

결함	Image 특징	1차 Algorithm	보강
Scratch	길고 가는 Contrast/Edge	Directional Filter, Line/Blob	Dark Field 다방향
Particle	국부 밝기/어두운 Blob	Threshold/Top-hat+Blob	Area/Circularity/CNR
Crack	가는 분기 Dark/bright Line	Ridge/Edge, Morphology	Skeleton/Length
Missing	Expected Feature 부재	Matching/Blob Count	Align 성공 전제
Stain	넓고 약한 Local Difference	Background/Reference Difference	Local 통계
Contamination	불규칙 Blob/Texture	Threshold/Texture/Difference	정상 Variation
Pattern Defect	기준 Pattern과 불일치	Align+Reference Difference	Registration Residual

3.4 Presence

단순 Count만 아니라 Expected ROI, Area/Position/Shape/Score를 함께 사용한다. Absence와 “검사 불능”을 분리한다. Align 실패나 빈 Image는 Missing NG가 아니라 System/Inspection Error다.

3.5 Position

Pattern Center, Blob Centroid, Edge Intersection 중 제품에서 반복 가능한 Datum을 선택한다. Blob Centroid는 밝기/Threshold 변화로 움직일 수 있고 Pattern Center는 Template Center Offset 정의가 필요하다.

3.6 치수

Binary Bounding Box보다 Gray Edge Profile의 Sub-pixel Edge Pair가 정밀 치수에 유리하다. Calibration, Distortion, Focus, 조명 Telecentricity와 Gauge R&R을 포함한다.

3.7 Scratch

방향성 Dark Field와 Line/Gradient가 일반적이다. Scratch 폭이 Optical PSF보다 작으면 Width 측정보다 Presence/Contrast Energy 검출로 정의할 수 있다. 방향별 사각지대를 Test한다.

3.8 Particle

Background를 제거한 Local Contrast Image에서 Blob을 분석한다. Area만으로 Dust, Texture, Sensor Hot Pixel을 구분하지 못하므로 Peak/Mean Contrast, Shape, 지속성, 위치를 사용한다.

3.9 Crack

Crack은 가늘고 분기하며 Contrast가 위치별로 달라질 수 있다. Ridge/Edge, 방향성 Filter, Closing/Skeleton을 사용할 수 있지만 Morphology가 길이와 연결성을 바꾼다. Raw Overlay와 End/Branch Point를 검증한다.

3.10 Stain/Contamination

넓고 약한 결함은 Global Threshold보다 Background Estimation, Reference Difference, Local Mean/Variance가 유리하다. 정상 Shading, Pattern, 조명 Drift가 False Positive를 만든다.

3.11 Pattern Defect

Reference와 Current Image를 정확히 Align한 뒤 Difference한다. 0.1 Pixel Registration Error도 강한 정상 Edge에 큰 Difference를 만든다. Edge 주변 Tolerance Band 또는 Distance-aware 비교를 검토한다.

3.12 Feature/Measurement/Judgement 분리

FORMULA · CALCULATION

Feature: edges/blobs/matches

Measurement: width=1.0025 mm, area=0.020 mm²

Judgement: $\min \leq \text{value} \leq \max \rightarrow \text{OK/NG}$

Spec 변경이 Image Processing을 바꾸지 않도록 분리한다.

3.13 Guard Band

측정 불확실도가 Spec에 비해 크면 경계값 판정이 흔들린다.

TEXT

Spec OK: 9.90~10.10 mm

Guard Band: 9.92~10.08 mm만 자동 OK

경계: Review/재검 정책

Guard Band는 품질/계측 합의와 Measurement System Analysis에 근거한다.

3.14 Deep Learning 적용 경계

전통 Vision이 유리:

- 명확한 Geometry/치수/Threshold
- 설명 가능한 Rule과 적은 Data

DL을 검토:

- 정상 Texture Variation이 크고 결함 모양이 다양
- Rule 조합이 과도하며 충분한 Label Data가 있음

DL도 조명/광학/좌표/Result Architecture를 대체하지 않는다.

4. 그림으로 이해하기

TEXT

Requirement → Visual Feature → Preprocess → Extract → Measure → Judge
 ↑ ↓
Physical Sample ← Review/Failure Analysis ← Result

5. 실제 검사 장비에서 어디에 사용하는가?

- 조립 유무/방향 판정
- Pad/Mark Position과 Align
- 외경/Gap/Width 측정
- Glass/Metal 표면 결함
- Pattern 누락/오염 Review
- 품질 통계와 공정 Feedback

6. 숫자로 이해하기

예제 1: Width

FORMULA · CALCULATION

Edge=(120.3,320.8) px, Scale=5 $\mu\text{m}/\text{px}$
Width=(320.8-120.3) $\times$ 5=1002.5 μm =1.0025 mm

예제 2: Particle Area

FORMULA · CALCULATION

Area=180 px², sx=4.8 μm/px, sy=5.1 μm/px
Area=180×4.8×5.1=4406.4 μm²=0.0044064 mm²

예제 3: Detection 성능

불량 200개 중 194 검출, 정상 1000개 중 12개 오검:

FORMULA · CALCULATION

Detection Rate=194/200=97%

False Positive Rate=12/1000=1.2%

예제 4: Angle Error 영향

100 mm 길이에서 0.05° 방향 Error의 끝단 횡오차:

FORMULA · CALCULATION

$100 \times \tan(0.05^\circ) \approx 0.0873 \text{ mm}$

7. C++ 구현

C++

```
#include <opencv2/opencv.hpp>
#include <algorithm>
#include <memory>
#include <stdexcept>
#include <string>
#include <vector>

enum class Verdict { Ok, Ng, Error };
struct Measurement {std::string name; double value, lower, upper; std::string unit;};
struct InspectionOutput {Verdict verdict; std::vector<Measurement> values; cv::Mat overlay; std::string reason;};

class IInspector {
public:
    virtual ~IInspector()=default;
    [[nodiscard]] virtual InspectionOutput Inspect(const cv::Mat& gray) const=0;
};

class PresenceByArea final: public IInspector {
public:
    PresenceByArea(double threshold, int minArea, int maxArea)
        : threshold_(threshold), minArea_(minArea), maxArea_(maxArea)
    {if(threshold<0 || threshold>255 || minArea<0 || minArea>maxArea) throw std::invalid_argument{"Invalid recipe"}};

    [[nodiscard]] InspectionOutput Inspect(const cv::Mat& gray) const override {
        if(gray.empty() || gray.type()!=CV_8UC1) return {Verdict::Error, {}, {}, "Invalid image"};
        cv::Mat binary; cv::threshold(gray, binary, threshold_, 255, cv::THRESH_BINARY);
        cv::Mat labels, stats, centroids;
        const int n=cv::connectedComponentsWithStats(binary, labels, stats, centroids, 8);
        int largest=0;
        for(int i=1; i<n; ++i) largest=std::max(largest, stats.at<int>(i, cv::CC_STAT_AREA));
        const bool ok=largest>=minArea_ && largest<=maxArea_;
        return {ok?Verdict::Ok:Verdict::Ng,
                {"largest_area", static_cast<double>(largest),
                 static_cast<double>(minArea_), static_cast<double>(maxArea_), "px^2"},
                {}, ok?"": "Presence area out of range"};
    }
private: double threshold_; int minArea_, maxArea_;
};
```

실제 구현에서는 ROI/Align/Calibration을 Engine이 준비하고 Inspector는 명시된 입력/Recipe로 Feature와 Measurement를 반환한다. Error와 제품 NG를 구분한다.

8. 실무에서 발생하는 문제

1. 결함 이름만으로 Algorithm 선택.
2. Align 실패를 Missing으로 판정.
3. Golden 한 장에 Overfit.
4. Morphology로 결함 크기 변경.

5. Threshold/조명 Drift로 Area 변동.
6. Detection과 Measurement 성능 혼동.
7. 정상 Variation 부족으로 False Positive 급증.

9. 흔한 오해

1. Scratch는 Canny 하나로 검사한다.
2. Particle은 Area만 보면 된다.
3. Reference Difference가 크면 결함이다.
4. Sub-pixel이면 실제 해상도가 높아진다.
5. 검출률만 높으면 좋은 검사다.
6. DL이면 조명/Align이 필요 없다.

10. 면접에서 나올 수 있는 질문

1. **Scratch Feature?** Line/Edge/방향성 Contrast/긴 Blob.
2. **Particle?** Local Threshold/Top-hat+Blob, Area/Contrast/Shape.
3. **Position?** Pattern/Edge/Blob Datum과 Calibration.
4. **Measurement/Judgement 분리 이유?** Spec 변경/추적/Review.
5. **Reference Difference 주의?** Registration/조명 Normal Variation.
6. **성능 평가?** Detection, False Positive, Bias, Repeatability, Cycle Time.

11. 실습 문제

1. 검사 10종을 Feature/Algorithm/조명/실패 원인 표로 만든다.
2. Particle Area/Contrast/Circularity Rule의 Confusion Matrix를 계산한다.
3. Reference Image를 0.1~1 Pixel 이동해 Difference False Defect를 측정한다.
4. Width 측정의 Bias/Repeatability/Guard Band를 설계한다.

Phase 13 미니 프로젝트

Presence, Width, Particle 세 Inspector를 동일 Interface로 구현하고 Measurement/Spec/Verdict/Error/Overlay를 JSON과 Image로 저장한다.

12. Chapter 핵심 요약

- 결함을 Image Feature로 번역해 Algorithm을 선택한다.
- 조명/Align/ROI가 Algorithm보다 선행한다.
- Detection과 Measurement/Judgement를 분리한다.
- Score/Area 하나가 아니라 여러 특징과 위치를 검증한다.

- ● 실제 Variation으로 검출률과 오검률을 평가한다.
- ● Error와 제품 NG를 구분한다.
- ● DL은 전통 Vision으로 분리가 어려운 Texture 문제에 제한적으로 검토한다.

다음 문서

16일차 B. Inspection Pipeline

16일차 B. Inspection Pipeline: Acquisition에서 Result까지

Phase 14 · 예상 학습 시간: 12~16시간 · 난이도: 중급→고급 · 중요도: ● 반드시 알아야 함

1. 이 Chapter에서 배우는 것

- ● Pipeline 단계별 책임과 Data Contract
- ● Acquisition/Preprocess/Align/ROI/Feature/Measure/Judge
- ● 제품 NG와 System Error 분리
- ● Recipe Snapshot과 Result Traceability
- ● Intermediate Image/Overlay/NG 저장
- ● Cycle Time과 비동기 저장
- ● Stage별 Timing/Quality Gate
- ● Testable Architecture

실무 목표

한 함수에 모든 OpenCV 호출을 섞지 않고 단계별 입력/출력과 실패 정책을 설계한다. 동일 Frame/Recipe/Calibration로 재현 가능한 Result를 만들고 UI/Review/CIM이 같은 확정 Result를 소비하게 한다.

2. 선수 지식

- 검사 알고리즘 선택
- Alignment
- Dynamic ROI
- Calibration

3. 핵심 개념

3.1 전체 Pipeline

TEXT

```
Image Acquisition
→ Validation
→ Preprocessing
→ Alignment
→ ROI Transform
→ Feature Extraction
→ Inspection/Measurement
→ Judgement
→ Result Finalize
→ Review/Storage/CIM
```

3.2 Acquisition

Frame Image와 Camera ID, Frame/Trigger/Product ID, Timestamp, Exposure/Gain, Buffer Ownership을 확정한다. 빈/불완전/오래된 Frame은 제품 NG가 아니라 Acquisition Error다.

3.3 Validation

- Image Type/Size/Channel
- Sequence/Product ID
- Recipe/Calibration 호환성
- Saturation/Mean 등 선택적 Quality Gate

입력 계약이 깨졌으면 Algorithm을 실행하지 않는다.

3.4 Preprocessing

검사에 필요한 Gray/Filter/Normalize/Difference를 생성한다. 원본은 불변으로 보존하고 Parameter와 중간 Image를 추적 가능하게 한다.

3.5 Alignment

Mark를 검출하고 X/Y/Theta, Score, Residual을 반환한다. Align 실패를 제품 Missing으로 바꾸지 않는다. 일부 검사만 Align 없이 가능한지 Recipe 정책을 명시한다.

3.6 ROI Transform

Reference ROI를 Current Pose로 변환하고 경계/Mask를 검증한다. 어떤 ROI가 잘렸는지 Inspector가 모르게 숨기지 않는다.

3.7 Feature Extraction

Threshold/Edge/Blob/Matching 등 Image에서 후보를 만든다. 이 단계는 가능하면 Spec 판정을 하지 않고 Feature와 품질 지표를 반환한다.

3.8 Measurement

Feature를 Pixel/실제 단위의 Width/Area/Position/Score로 변환한다. Calibration ID와 단위를 Measurement에 포함한다.

3.9 Judgement

Recipe Spec, Guard Band, Invalid 상태로 OK/NG/Review/Error를 결정한다. 여러 항목의 Overall 규칙도 명시한다.

3.10 Result Finalize

Result는 확정 후 불변으로 취급한다.

- 모든 항목 Measurement/Spec/Verdict
- Align/Calibration/Recipe Version
- Error/Warning
- Timing
- Raw/NG/Overlay 경로

3.11 Review/Storage/CIM

동일 Result를 서로 다른 Consumer가 사용한다. 비동기 저장은 Cycle Time을 줄이지만 Queue Limit, Backpressure, 종료 시 Flush와 실패 Alarm이 필요하다.

3.12 Fail-fast와 Partial Result

Camera Frame Error처럼 이후 결과가 무의미하면 Fail-fast한다. 한 검사 항목만 실패하고 나머지가 유효하면 항목별 Invalid와 Overall 정책을 사용한다. 임의 기본값 0 으로 채우지 않는다.

3.13 Determinism/Reproducibility

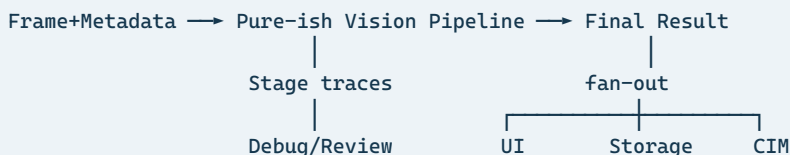
동일 Raw Image, Recipe Snapshot, Calibration/Algorithm Version이면 동일 Result가 나와야 한다. Thread 순서, Random Seed, Adaptive Parameter와 외부 상태를 통제한다.

3.14 Timing Budget

각 Stage의 Average뿐 아니라 P95/P99/Max를 본다. 저장/압축/Network를 검사 Thread에서 동기 실행하지 않되 Memory Queue가 무한 증가하지 않게 한다.

4. 그림으로 이해하기

TEXT



FORMULA · CALCULATION

NG = valid inspection says product is bad

Error = system cannot produce a trustworthy judgement

5. 실제 검사 장비에서 어디에 사용하는가?

- Product ID와 Frame ID 결합
- Mark Align 후 다수 ROI 검사
- 항목별 측정과 Overall 판정
- NG Image/Overlay Review
- CIM/MES 결과와 장비 분류 동기화
- 성능/오류 Monitoring

6. 숫자로 이해하기

예제 1: Cycle Time

FORMULA · CALCULATION

Acquire 15 + Validate 2 + Preprocess 12 + Align 35
+ ROI 3 + Inspect 48 + Finalize 5 = 120 ms

목표 150 ms이면 30 ms Margin이다. 동기 Image 저장 45 ms를 추가하면 초과한다.

예제 2: Queue Memory

12 MiB Raw+Overlay 묶음을 100개 Queue하면 약 1.17 GiB다. Queue 20개 제한이면 약 240 MiB이며 초과 시 NG 우선 /Backpressure/Error 정책이 필요하다.

예제 3: Overall

10항목 중 8 OK, 1 NG, 1 Invalid라면 단순 Majority OK가 아니다. Recipe가 Any NG→NG , Any required Invalid→Error 라면 Overall Error이며 NG 정보도 보존한다.

예제 4: Throughput

처리 평균 80 ms지만 P99가 180 ms이고 Input 간격이 100 ms면 순간 Queue가 증가한다. Average만으로 안정성을 판단하지 않는다.

7. C++ 구현

CPP · 1/2

```
#include <opencv2/opencv.hpp>
#include <algorithm>
#include <chrono>
#include <cstdint>
#include <stdexcept>
#include <string>
#include <vector>

enum class Verdict { Ok, Ng, Review, Error };

struct Frame final {cv::Mat image;std::uint64_t frameId{};std::string productId,cameraId{};
struct Recipe final {std::string id;std::uint32_t version{};cv::Rect referenceRoi;double
    threshold{},minArea{},maxArea{};};
struct AlignPose final {double tx{},ty{},thetaDegrees{},score{},rmsResidual{};bool valid{};};
struct Measurement final {std::string name;double value{},lower{},upper{};std::string unit;Verdict
    verdict{Verdict::Error};};
struct InspectionResult final {
    std::uint64_t frameId{};std::string productId,recipeId,error;
    std::uint32_t recipeVersion{};Verdict overall{Verdict::Error};
    AlignPose align;std::vector<Measurement> measurements;cv::Mat overlay;
    double elapsedMs{};
};

class IAligner {public:virtual ~IAligner()=default;virtual AlignPose Align(const cv::Mat&)const=0;};

class InspectionEngine final {
public:
    explicit InspectionEngine(const IAligner& aligner):aligner_(aligner){}

    [[nodiscard]] InspectionResult Inspect(const Frame& frame,const Recipe& recipe) const {
        const auto start=std::chrono::steady_clock::now();
        InspectionResult out;out.frameId=frame.frameId;out.productId=frame.productId;
        out.recipeId=recipe.id;out.recipeVersion=recipe.version;
        try {
            Validate(frame,recipe);
            cv::Mat gray;
            if(frame.image.channels()==1)gray=frame.image;
            else cv::cvtColor(frame.image,gray,cv::COLOR_BGR2GRAY);
            out.align=aligner_.Align(gray);
            if(!out.align.valid)throw std::runtime_error{"Alignment failed"};
        }
```

CPP · 2/2

```
// Simplified translation-only ROI; production code applies X/Y/Theta polygon transform.
const cv::Rect roi=recipe.referenceRoi+cv::Point{
    cvRound(out.align.tx),cvRound(out.align.ty)};
const cv::Rect bounds{0,0,gray.cols,gray.rows};
if((roi&bounds)!=roi)throw std::runtime_error{"ROI leaves image"};
cv::Mat binary;cv::threshold(gray(roi),binary,recipe.threshold,255,cv::THRESH_BINARY);
cv::Mat labels,stats,centroids;
const int count=cv::connectedComponentsWithStats(binary,labels,stats,centroids,8);
int largest=0;for(int i=1;i<count;++i)largest=std::max(largest,stats.at<int>(i,cv::CC_STAT_AREA));
const bool ok=largest>=recipe.minArea&&largest<=recipe.maxArea;
out.measurements.push_back({"largest_area",static_cast<double>(largest),
    recipe.minArea,recipe.maxArea,"px^2",ok?Verdict::Ok:Verdict::Ng});
out.overall=ok?Verdict::Ok:Verdict::Ng;
} catch(const std::exception& e) {out.overall=Verdict::Error;out.error=e.what();}
out.elapsedMs=std::chrono::duration<double,std::milli>(
    std::chrono::steady_clock::now()-start).count();
return out;
}
private:
static void Validate(const Frame& f,const Recipe& r){
    if(f.image.empty())throw std::invalid_argument{"Empty image"};
    if(r.id.empty()||r.referenceRoi.area()<=0||r.minArea>r.maxArea||
        r.threshold<0||r.threshold>255)throw std::invalid_argument{"Invalid recipe"};
}
const IAligner& aligner_;
};
```

예제 ROI는 코드 복잡도를 제한하기 위해 Translation만 적용한다. 실제 Engine은 Polygon X/Y/Theta Transform을 사용해야 한다. 이 제한을 숨기지 않는다.

Unit Test 범위

- Empty/잘못된 Type/Recipe → Error
- Align 실패 → Error, Missing NG 아님
- ROI 경계 초과 → Error
- Spec 내부/외부 → OK/NG
- 동일 Input/Recipe → 동일 Measurement
- 저장 실패가 검사 Result를 변조하지 않음

8. 실무에서 발생하는 문제

1. Algorithm 함수가 Recipe Global 상태를 직접 읽음.
2. 검사 중 UI가 Recipe를 변경.
3. NG와 Error가 bool false 하나로 표현.
4. Buffer 재사용 후 비동기 저장이 깨진 Image 읽음.
5. 저장 Queue 무한 증가.
6. UI와 CIM이 다른 판정 로직 사용.
7. 중간 Image가 없어 NG 원인 재현 불가.
8. P99 지연과 Timeout 무시.

9. 흔한 오해

1. Pipeline은 OpenCV 함수 호출 순서만 뜻한다.
2. 검사 항목 NG와 Align Error는 같다.
3. 비동기화하면 무조건 빨라진다.
4. Result는 Overall bool이면 충분하다.
5. Recipe Version 없이 Image만 저장해도 재현된다.
6. 평균 처리 시간이 Cycle Time보다 작으면 안정적이다.

10. 면접에서 나올 수 있는 질문

1. **Pipeline 단계?** Acquisition→Preprocess→Align→ROI→Feature→Measure→Judge→Result.
2. **Measurement/Judgement 분리?** Spec 변경, 추적성과 재판정.
3. **NG/Error 차이?** 유효한 제품 불량과 신뢰 판정 불가.
4. **Recipe Snapshot?** 검사 중 Parameter 일관성과 재현성.
5. **비동기 저장 주의?** Buffer 수명, Queue Limit, Backpressure, 실패 Alarm.
6. **성능 평가?** Stage별 P50/P95/P99/Max와 Queue/Memory.

11. 실습 문제

1. 현재 장비 기능을 Pipeline Stage와 Data Contract에 매핑한다.
2. 각 Stage의 Error/NG/Warning 정책표를 만든다.
3. 150 ms Cycle Time Budget과 P99 목표를 배분한다.
4. Recipe 변경/Camera Timeout/저장 실패 Integration Test를 작성한다.

Phase 14 미니 프로젝트

파일 Image Source, Fake Aligner, 3 Inspector, Result Writer를 조합해 Deterministic Pipeline과 Stage Timing/Intermediate Image/JSON Result를 구현한다.

12. Chapter 핵심 요약

- Pipeline은 단계별 책임과 Data Contract다.
- 원본/Recipe Snapshot/Calibration Version을 고정한다.
- Align/ROI 실패와 제품 NG를 분리한다.
- Feature→Measurement→Judgement를 분리한다.
- Final Result를 UI/Storage/CIM의 단일 Source로 사용한다.
- 비동기 저장은 Queue/Buffer 수명과 실패 정책이 필요하다.
- Average가 아니라 Tail Latency와 Memory를 본다.

다음 학습 예고

17일차에는 Recipe/Parameter/Result Data Model과 실제 C++ 검사 SW Architecture를 설계한다.